

1.
https://blog.csdn.net/v_JULY_v/article/details/133619540

通透理解FlashAttention(含其2代和第3代): 全面降低显存读写、加快计算速度

2.
<https://zhuanlan.zhihu.com/p/669926191>

图解大模型计算加速系列: FlashAttention V1, 从硬件到计算逻辑

3.
<https://zhuanlan.zhihu.com/p/691067658>

图解大模型计算加速系列: Flash Attention V2, 从原理到并行计算

```
//-----  
// fa 1  
//-----
```

减少 HBM 读写: 标准注意力需读写 S_h, P_h , FlashAttention 在 SRAM 完成。为啥? 请举例说明

感谢你的提问! 你关注 **FlashAttention v1** 如何减少 GPU 高带宽内存 (HBM) 读写, 指出标准注意力机制需要多次读写注意力分数矩阵 (S_h) 和概率矩阵 (P_h) , 而 FlashAttention 在 SRAM (片上内存) 完成计算。你要求解释其原因并通过具体例子说明。我将详细分析标准注意力与 FlashAttention 在内存访问上的差异, 解释为什么 FlashAttention 能在 SRAM 完成计算, 结合矩阵、数字和示例展示其优化机制。参考资料包括 FlashAttention 论文 (Dao et al., 2022)、PyTorch 实现和 GPU 内存架构文献。

一、背景: HBM 与 SRAM 在 GPU 计算中的作用

1. **GPU 内存层次**

- **HBM (高带宽内存)**:

- 位于 GPU 外部, 容量大 (例如, NVIDIA A100 有 40GB 或 80GB HBM3)。
- 带宽高 ($\sim 2\text{TB/s}$), 但访问延迟较高 ($\sim 100\text{ns}$)。
- 用于存储大矩阵 (如 $(S_h, P_h \in \mathbb{R}^{B \times H \times S \times S})$), 但频繁读写会导致性能瓶颈。

- **SRAM (片上内存, 寄存器或共享内存)**:

- 位于 GPU 芯片内部, 容量小 (例如, A100 每个 SM 有 $\sim 192\text{KB}$ 共享内存, 总 SRAM $\sim 10\text{MB}$)。
- 访问速度快 ($\sim 1\text{ns}$), 适合小块数据的高速计算。

- 容量限制要求数据分块处理。

2. **标准注意力的 HBM 读写问题**

- **标准 MHSA 计算**:

- 输入: $(X \in \mathbb{R}^{B \times S \times D})$, 投影到 $(Q, K, V \in \mathbb{R}^{B \times H \times S \times D_h})$, $(D_h = D/H)$ 。
- 注意力分数: $(S_h = \frac{Q_h K_h^T}{\sqrt{D_h}} \in \mathbb{R}^{B \times H \times S \times S})$ 。
- 概率: $(P_h = \text{Softmax}(S_h) \in \mathbb{R}^{B \times H \times S \times S})$ 。
- 输出: $(A_h = P_h V_h \in \mathbb{R}^{B \times H \times S \times D_h})$ 。

- **HBM 读写**:

- **写**: 计算 (S_h) 并写入 HBM ($\sim BHS^2$ 字节)。
- **读/写**: Softmax 读取 (S_h) , 计算 (P_h) , 写入 HBM。
- **读**: 读取 (P_h, V_h) , 计算 (A_h) , 写入 HBM。
- **问题**:
 - (S_h, P_h) 占用 $(O(BHS^2))$ 内存, 例如, $(B=1, H=32, S=2048)$, FP16 下 $(S_h, P_h \approx 0.5)$ GB。
 - 多次 HBM 读写 (至少 3 次: 写 (S_h) , 读/写 (P_h) , 读 (P_h/V_h)), 增加延迟 ($\sim 0.5\text{GB} \times 3 / 2\text{TB/s} \approx 0.75\text{ms}$)。

3. **FlashAttention 的优化**

- **核心思想**:

- **块状计算**: 将 (Q, K, V) 分块 (块大小 $(S_r \parallel S)$), 每次处理小矩阵 (如 $(S_{ij} \in \mathbb{R}^{B \times H \times S_r \times S_r})$), 适配 SRAM 容量。
- **在线 Softmax**: 在块内计算 Softmax, 避免存储整个 (P_h) 。
- **内核融合**: 将 $(Q K^T)$ 、Softmax 和 $(P_h V_h)$ 融合到单个 CUDA 内核, 数据保留在 SRAM, 减少 HBM 访问。
- **为什么在 SRAM 完成**:
 - SRAM 容量小但速度快, 适合处理小块数据 (例如, $(S_r=128)$, $\sim 128\text{KB}$)。
 - 块状计算确保每次只加载 $(Q_i \in \mathbb{R}^{B \times H \times S_r \times D_h})$, $(K_j, V_j \in \mathbb{R}^{B \times H \times S_r \times D_h})$, 内存占用 $(O(BHS_r D_h))$, 远小于 $(O(BHS^2))$ 。
 - 在线 Softmax 直接计算 $(A_{ij} = \text{Softmax}(S_{ij}) V_j)$, 无需存储 (S_{ij}, P_{ij}) 到 HBM。

二、具体示例: FlashAttention v1 减少 HBM 读写

1. **场景**

- **输入**:

- $(X \in \mathbb{R}^{1 \times 8 \times 4})$ ($(B=1, S=8, D=4)$) 。
- 头数: $(H=2, D_h = 4/2 = 2)$ 。
- **参数**:
- $(W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4})$ 。
- **块大小**:
- 序列分块: $(S_r=4)$, 2 块 (0-3, 4-7) 。
- **硬件**: NVIDIA A100, HBM ~40GB, SRAM ~10MB/SM。
- **输入矩阵**:
- $$\begin{aligned} & \backslash[\\ & X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4} \\ & \backslash] \end{aligned}$$
- **权重** (简化, 单位矩阵) :
- $$\begin{aligned} & \backslash[\\ & W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ & \backslash] \end{aligned}$$
- #### 2. **标准注意力: HBM 读写**
- **计算**:
- 投影: $(Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4})$ 。
- 头 0: $(Q_0, K_0, V_0 = X[:, :, 0:2] \in \mathbb{R}^{1 \times 8 \times 2})$ 。
- 注意力分数:
- $$\begin{aligned} & \backslash[\\ & S_0 = \frac{Q_0 K_0^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 8} \\ & \backslash] \end{aligned}$$
- Softmax: $(P_0 = \text{Softmax}(S_0) \approx \begin{bmatrix}$

$$\begin{bmatrix} 0.25 & 0 & 0.25 & 0 & 0.25 & 0 & 0.25 & 0 \\ 0 & 0.25 & 0 & 0.25 & 0 & 0.25 & 0 & 0.25 \\ \vdots & & & & & & & \end{bmatrix}.$$

- 输出: $(A_0 = P_0 V_0 \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ \vdots & \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 2})$.

- **HBM 读写**:

- 写 (S_0) : $(1 \times 8 \times 8 \times 2 = 128)$ bytes.
- 读 (S_0) , 写 (P_0) : $128 + 128$ bytes.
- 读 (P_0, V_0) : $128 + (1 \times 8 \times 2 \times 2 = 32)$ bytes.
- 总计: ~ 416 bytes 读写, HBM 带宽 2TB/s, 延迟 $\sim 0.2\mu\text{s}$.
- 内存: 存储 $(S_0, P_0 \approx 256)$ bytes.

3. **FlashAttention v1: SRAM 计算**

- **分块**:

- $(Q_0, K_0, V_0 \in \mathbb{R}^{1 \times 8 \times 2})$, 分 2 块 $(S_r=4)$:
 - 块 1: $(Q_0[0:4], K_0[0:4], V_0[0:4])$.
 - 块 2: $(Q_0[4:8], K_0[4:8], V_0[4:8])$.

- **计算**:

- **块 1-1** $(Q_0[0:4], K_0[0:4], V_0[0:4])$:

$$\begin{aligned} S_{11} &= \frac{Q_0[0:4] K_0[0:4]^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \\ P_{11} &= \text{Softmax}(S_{11}) \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix} \\ A_{11} &= P_{11} V_0[0:4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

- **块 1-2** $(Q_0[0:4], K_0[4:8], V_0[4:8])$:

```

\l
S_{12} = \begin{bmatrix}
1 & 0 & 1 & 0 \\
0 & 1 & 0 & 1 \\
1 & 0 & 1 & 0 \\
0 & 1 & 0 & 1
\end{bmatrix}, \quad P_{12} \approx \begin{bmatrix}
0.5 & 0 & 0.5 & 0 \\
0 & 0.5 & 0 & 0.5 \\
0.5 & 0 & 0.5 & 0 \\
0 & 0.5 & 0 & 0.5
\end{bmatrix}
\l
\l
A_{12} = P_{12} V_0[4:8] \approx \begin{bmatrix}
1 & 0 \\
0 & 1 \\
1 & 0 \\
0 & 1
\end{bmatrix}
\l

```

- **在线 Softmax**:

- 归一化: $\l(m_i = \max(S_{11}, S_{12}) \approx 1 \r), \l(l_i = \sum \exp(S_{11} - 1) + \exp(S_{12} - 1) \approx 4 \r).$

- 合并: $\l(A_i[0:4] = \frac{\exp(S_{11} - 1) V_0[0:4] + \exp(S_{12} - 1) V_0[4:8]}{4} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \r).$

- **块 2**: 类似计算 $\l(Q_0[4:8] \r).$

- **HBM 读写**:

- 加载: $\l(Q_0[0:4], K_0[0:4], V_0[0:4] \approx 3 \times 1 \times 4 \times 2 \times 2 = 24 \r)$ bytes/块。

- 计算: $\l(S_{ij}, P_{ij}, A_{ij} \r)$ 在 SRAM (~32 bytes), 不写入 HBM。

- 写: 仅写 $\l(A_i[0:4] \approx 16 \r)$ bytes/块到 HBM。

- 总计: ~40 bytes 读写/块, 2 块 ~80 bytes, 远低于标准注意力的 416 bytes。

- **内存**:

- SRAM: 存储 $\l(S_{ij} \in \mathbb{R}^{1 \times 4 \times 4} \approx 32 \r)$ bytes。

- HBM: 仅存储 $\l(A_i \approx 32 \r)$ bytes (最终输出)。

4. **流程图 (文本形式)**

- **标准注意力**:

...

```

[X: 1×8×4] → [Q, K, V: 1×8×4] → [S_h: 1×2×8×8] → [P_h: 1×2×8×8] → [A_h: 1×2×8×2]
|           |           |           |           |
[HBM 写]    [HBM 读/写]  [HBM 读/写: 256B]  [HBM 读: 256B]  [HBM 写: 32B]

```

```

... [内存: 32B] [内存: 32B] [内存: 256B] [内存: 256B] [内存: 32B]
...
- **FlashAttention**:
...
[X: 1×8×4] → [Q, K, V: 1×8×4] → [Block Q[0:4], K[0:4], V[0:4]] → [S_{11}, P_{11}, A_{11}] → [A: 1×8×4]
|           |           |           |           |
[HBM 读: 32B] [HBM 读: 24B/块] [SRAM: 32B] [HBM 写: 16B/块] [HBM: 32B]
[内存: 32B] [内存: 24B/块] [内存: 32B] [内存释放] [输出]
...

```

三、为什么 FlashAttention 在 SRAM 完成

1. **块状计算**:

- 分块 ($S_r=4$) 将 $(S_h \in \mathbb{R}^{1 \times 8 \times 8})$ 拆成 $(S_{ij} \in \mathbb{R}^{1 \times 4 \times 4})$, 内存从 128 bytes 降到 32 bytes, 适配 SRAM (~192KB/SM)。
- 每次加载小块 (Q_i, K_j, V_j) , 在 SRAM 计算 (S_{ij}, P_{ij}, A_{ij}) 。

2. **在线 Softmax**:

- 避免存储 (P_h) , 通过归一化因子 (m_i, l_i) 在 SRAM 累积:

$$\begin{aligned} m_i^{(j)} &= \max(m_i^{(j-1)}, \max(S_{ij})), \quad l_i^{(j)} = l_i^{(j-1)} \exp(m_i^{(j-1)} - m_i^{(j)}) \\ &+ \sum \exp(S_{ij} - m_i^{(j)}) \\ & \end{aligned}$$
- 直接输出 (A_{ij}) , 无需写 (S_{ij}, P_{ij}) 到 HBM。

3. **内核融合**:

- 融合 (QK^T) , Softmax, (PV) 到单个 CUDA 内核, 数据保留在 SRAM, 减少 HBM 读写 (~80 bytes vs 416 bytes)。

4. **HBM 访问**:

- 仅读 (Q_i, K_j, V_j) (~24 bytes/块), 写 (A_i) (~16 bytes/块), 总计 ~80 bytes, 远低于标准注意力的 416 bytes。

4. **LLaMA-3.1-8B 示例**

- **场景**:

- $(B=16, S=128, D=4096, H=32, D_h=128)$ 。
- 块大小: $(S_r=64)$ 。

- **标准注意力**:

- HBM: $(S_h, P_h \in \mathbb{R}^{16 \times 32 \times 128 \times 128}) \approx 8$ GB。
- 读写: ~24GB (写 (S_h) , 读/写 (P_h) , 读 (P_h/V_h)), 延迟 ~12ms (2TB/s)。

- **FlashAttention**:

- SRAM: $(S_{ij} \in \mathbb{R}^{16 \times 32 \times 64 \times 64}) \approx 2$ MB。

- HBM: 读 $\mathcal{O}(Q_i, K_j, V_j) \approx 1$ MB/块, 写 $\mathcal{O}(A_i) \approx 0.5$ MB/块, 总计 ~3MB, 延迟 ~1.5 μ s.
- 内存: 降到 ~0.04% (2MB vs 8GB), 速度提升 ~2x.

四、注意事项

- **SRAM 限制**: 块大小 $\mathcal{O}(S_r)$ 受 SRAM 容量限制 (A100 ~10MB), 需调整以适配。
- **硬件支持**: FlashAttention 依赖 GPU 的快速 SRAM 和 CUDA 优化。
- **Grok 3** (<https://x.ai/api>): Grok 3 的长上下文 (>100k tokens) 需 FlashAttention 减少 HBM 访问, 提升推理效率。

五、总结

- **为什么在 SRAM**:
 - 块状计算将 $\mathcal{O}(S_h, P_h)$ 拆成小块 (~32 bytes), 适配 SRAM。
 - 在线 Softmax 和内核融合避免存储 $\mathcal{O}(S_h, P_h)$, 减少 HBM 读写 (~80 bytes vs 416 bytes)。
- **示例**: $\mathcal{O}(S=8, S_r=4)$, FlashAttention 在 SRAM 计算 $\mathcal{O}(S_{ij}, P_{ij})$, HBM 访问降到 1/5。
- **效果**: 内存从 $\mathcal{O}(S^2)$ 降到 $\mathcal{O}(S)$, 速度提升 2-4 倍。

如果你需要 Chart.js 流程图、更复杂示例或与 FSDP/张量并行的对比, 请告诉我, 我可以进一步定制!

• HBM 读写:

• 为什么在 SRAM 完成:

- SRAM 容量小但速度快, 适合处理小块数据 (例如, $S_r = 128$, $\sim 128\text{KB}$) 。
- 块状计算确保每次只加载 $Q_i \in \mathbb{R}^{B \times H \times S_r \times D_h}$, $K_j, V_j \in \mathbb{R}^{B \times H \times S_r \times D_h}$, 内存占用 $O(BHS_r D_h)$, 远小于 $O(BHS^2)$ 。
- 在线 Softmax 直接计算 $A_{ij} = \text{Softmax}(S_{ij})V_j$, 无需存储 S_{ij}, P_{ij} 到 HBM。

二、具体示例: FlashAttention v1 减少 HBM 读写

1. 场景

- 输入:
 - $X \in \mathbb{R}^{1 \times 8 \times 4}$ ($B = 1, S = 8, D = 4$) 。
 - 头数: $H = 2, D_h = 4/2 = 2$ 。
- 参数:
 - $W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4}$ 。
- 块大小:
 - 序列分块: $S_r = 4$, 2 块 (0-3, 4-7) 。
- 硬件: NVIDIA A100, HBM $\sim 40\text{GB}$, SRAM $\sim 10\text{MB/SM}$ 。
- 输入矩阵:

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}$$

- 在线 Softmax 直接计算 $A_{ij} = \text{Softmax}(S_{ij})V_j$, 无需存储 S_{ij}, P_{ij} 到 HBM。



Copy

- 硬件：NVIDIA A100, HBM ~40GB, SRAM ~10MB/SM。

- 输入矩阵：

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}$$

- 权重（简化，单位矩阵）：

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. 标准注意力：HBM 读写

- 计算：

- 投影： $Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4}$.
- 头 0: $Q_0, K_0, V_0 = X[:, :, 0 : 2] \in \mathbb{R}^{1 \times 8 \times 2}$.
- 注意力分数：

$$S_0 = \frac{Q_0 K_0^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 8}$$

2. 标准注意力：HBM 读写

• 计算：

- 投影： $Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4}$.
- 头 0： $Q_0, K_0, V_0 = X[:, :, 0 : 2] \in \mathbb{R}^{1 \times 8 \times 2}$.
- 注意力分数：

$$S_0 = \frac{Q_0 K_0^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 8}$$

- Softmax： $P_0 = \text{Softmax}(S_0) \approx \begin{bmatrix} 0.25 & 0 & 0.25 & 0 & 0.25 & 0 & 0.25 & 0 \\ 0 & 0.25 & 0 & 0.25 & 0 & 0.25 & 0 & 0.25 \\ \dots & & & & & & & \end{bmatrix}$.

- 输出： $A_0 = P_0 V_0 \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ \dots & \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 2}$.

• HBM 读写：

- 写 S_0 : $1 \times 8 \times 8 \times 2 = 128$ bytes.
- 读 S_0 , 写 P_0 : $128 + 128$ bytes.
- 读 P_0, V_0 : $128 + 1 \times 8 \times 2 \times 2 = 32$ bytes.
- 总计： ~ 416 bytes 读写，HBM 带宽 2TB/s，延迟 $\sim 0.2 \mu\text{s}$.
- 内存：存储 $S_0, P_0 \approx 256$ bytes.

- 读 P_0, V_0 : $128 + 1 \times 8 \times 2 \times 2 = 32$ bytes.
- 总计: ~ 416 bytes 读写, HBM 带宽 2TB/s, 延迟 $\sim 0.2\mu\text{s}$.
- 内存: 存储 $S_0, P_0 \approx 256$ bytes.

3. FlashAttention v1: SRAM 计算

- 分块:
 - $Q_0, K_0, V_0 \in \mathbb{R}^{1 \times 8 \times 2}$, 分 2 块 ($S_r = 4$) :
 - 块 1: $Q_0[0 : 4], K_0[0 : 4], V_0[0 : 4]$.
 - 块 2: $Q_0[4 : 8], K_0[4 : 8], V_0[4 : 8]$.
- 计算:
 - 块 1-1 ($Q_0[0 : 4], K_0[0 : 4], V_0[0 : 4]$):

$$S_{11} = \frac{Q_0[0 : 4]K_0[0 : 4]^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$P_{11} = \text{Softmax}(S_{11}) \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{11} = P_{11}V_0[0 : 4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 块 1-2 ($Q_0[0 : 4], K_0[4 : 8], V_0[4 : 8]$):

- 块 1-2 ($Q_0[0 : 4], K_0[4 : 8], V_0[4 : 8]$):

$$S_{12} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{12} \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{12} = P_{12}V_0[4 : 8] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 在线 Softmax:

- 归一化: $m_i = \max(S_{11}, S_{12}) \approx 1, l_i = \sum \exp(S_{11} - 1) + \exp(S_{12} - 1) \approx 4.$

- 合并: $A_i[0 : 4] = \frac{\exp(S_{11}-1)V_0[0:4] + \exp(S_{12}-1)V_0[4:8]}{4} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$

- 块 2: 类似计算 $Q_0[4 : 8]$.

HBM 读写:

- 加载: $Q_0[0 : 4], K_0[0 : 4], V_0[0 : 4] \approx 3 \times 1 \times 4 \times 2 \times 2 = 24$ bytes/块。
- 计算: S_{ij}, P_{ij}, A_{ij} 在 SRAM (~32 bytes), 不写入 HBM。
- 写: 仅写 $A_i[0 : 4] \approx 16$ bytes/块到 HBM。
- 总计: ~40 bytes 读写/块, 2 块 ~80 bytes, 远低于标准注意力的 416 bytes。

内存:

- SRAM: 存储 $S_{ij} \in \mathbb{R}^{1 \times 4 \times 4} \approx 32$ bytes。



• 内存:

Copy

- SRAM: 存储 $S_{ij} \in \mathbb{R}^{1 \times 4 \times 4} \approx 32$ bytes。
- HBM: 仅存储 $A_i \approx 32$ bytes (最终输出)。

4. 流程图 (文本形式)

- 标准注意力:

text

✕ Collapse

≡ Wrap

📄 Copy

```
[X: 1×8×4] → [Q, K, V: 1×8×4] → [S_h: 1×2×8×8] → [P_h: 1×2×8×8] → [A_h: 1×2×8×2]
|           |           |           |           |
[HBM 写]    [HBM 读/写]  [HBM 读/写: 256B]  [HBM 读: 256B]  [HBM 写: 32B]
[内存: 32B]  [内存: 32B]  [内存: 256B]      [内存: 256B]      [内存: 32B]
```

- FlashAttention:

text

✕ Collapse

≡ Wrap

📄 Copy

```
K, V: 1×8×4] → [Block Q[0:4], K[0:4], V[0:4]] → [S_{11}, P_{11}, A_{11}] → [A: 1×8×4]
|           |           |           |           |
IBM 读: 24B/块  [SRAM: 32B]  [HBM 写: 16B/块]  [HBM: 32B]
3存: 24B/块    [内存: 32B]   [内存释放]        [输出]
```

三、为什么 FlashAttention 在 SRAM 完成

1. 块状计算:

- 分块 ($S_r = 4$) 将 $S_h \in \mathbb{R}^{1 \times 8 \times 8}$ 拆成 $S_{ij} \in \mathbb{R}^{1 \times 4 \times 4}$, 内存从 128 bytes 降到 32 bytes, 适配 SRAM (~192KB/SM)。



三、为什么 FlashAttention 在 SRAM 完成

1. 块状计算：

- 分块 ($S_r = 4$) 将 $S_h \in \mathbb{R}^{1 \times 8 \times 8}$ 拆成 $S_{ij} \in \mathbb{R}^{1 \times 4 \times 4}$ ，内存从 128 bytes 降到 32 bytes，适配 SRAM (~192KB/SM)。
- 每次加载小块 Q_i, K_j, V_j ，在 SRAM 计算 S_{ij}, P_{ij}, A_{ij} 。

2. 在线 Softmax：

- 避免存储 P_h ，通过归一化因子 m_i, l_i 在 SRAM 累积：

$$m_i^{(j)} = \max(m_i^{(j-1)}, \max(S_{ij})), \quad l_i^{(j)} = l_i^{(j-1)} \exp(m_i^{(j-1)} - m_i^{(j)}) + \sum \exp(S_{ij} - m_i^{(j)})$$

- 直接输出 A_{ij} ，无需写 S_{ij}, P_{ij} 到 HBM。

3. 内核融合：

- 融合 QK^T , Softmax, PV 到单个 CUDA 内核，数据保留在 SRAM，减少 HBM 读写 (~80 bytes vs 416 bytes)。

4. HBM 访问：

- 仅读 Q_i, K_j, V_j (~24 bytes/块)，写 A_i (~16 bytes/块)，总计 ~80 bytes，远低于标准注意力的 416 bytes。

4. LLaMA-3.1-8B 示例

• 场景：

- $B = 16, S = 128, D = 4096, H = 32, D_h = 128$.
- 块大小: $S_r = 64$.

• 标准注意力：

- 标准注意力：
 - HBM: $S_h, P_h \in \mathbb{R}^{16 \times 32 \times 128 \times 128} \approx 8\text{GB}$.
 - 读写: $\sim 24\text{GB}$ (写 S_h , 读/写 P_h , 读 P_h/V_h) , 延迟 $\sim 12\text{ms}$ (2TB/s) 。
- FlashAttention:
 - SRAM: $S_{ij} \in \mathbb{R}^{16 \times 32 \times 64 \times 64} \approx 2\text{MB}$.
 - HBM: 读 $Q_i, K_j, V_j \approx 1\text{MB/块}$, 写 $A_i \approx 0.5\text{MB/块}$, 总计 $\sim 3\text{MB}$, 延迟 $\sim 1.5\mu\text{s}$.
 - 内存: 降到 $\sim 0.04\%$ (2MB vs 8GB) , 速度提升 $\sim 2\text{x}$.

四、注意事项

- SRAM 限制: 块大小 S_r 受 SRAM 容量限制 (A100 $\sim 10\text{MB}$) , 需调整以适配。
- 硬件支持: FlashAttention 依赖 GPU 的快速 SRAM 和 CUDA 优化。
- Grok 3: Grok 3 的长上下文 ($>100\text{k tokens}$) 需 FlashAttention 减少 HBM 访问, 提升推理效率。

五、总结

- 为什么在 SRAM:
 - 块状计算将 S_h, P_h 拆成小块 ($\sim 32\text{ bytes}$) , 适配 SRAM。
 - 在线 Softmax 和内核融合避免存储 S_h, P_h , 减少 HBM 读写 ($\sim 80\text{ bytes vs }416\text{ bytes}$) 。
- 示例: $S = 8, S_r = 4$, FlashAttention 在 SRAM 计算 S_{ij}, P_{ij} , HBM 访问降到 1/5。
- 效果: 内存从 $O(S^2)$ 降到 $O(S)$, 速度提升 2-4 倍。

```
//-----  
//. fav1 backward propagation  
//-----
```

请用一个具体例子说明flash attention v1, 反向propagation, 有矩阵, 数据, 流程图, 例子

感谢你的提问! 你要求通过一个具体例子详细说明 **FlashAttention v1** 在**反向传播**

(backward propagation) 中的工作原理, 包括矩阵、数据、流程图和端到端示例。

FlashAttention v1 是一种高效的注意力机制实现, 通过块状计算、在线 Softmax 和内核融合优化了 Transformer 多头自注意力 (MHSA) 的内存和计算效率, 特别是在反向传播中减少了高带宽内存 (HBM) 读写。我将通过一个简化示例, 结合矩阵、数字和文本形式的流程图, 清晰展示 FlashAttention v1 的反向传播流程, 解释其优化机制, 并与标准注意力机制对比。参考资料包括 FlashAttention 论文 (Dao et al., 2022)、PyTorch 实现和 GPU 内存架构文献。

一、背景: FlashAttention v1 与反向传播

1. **标准注意力反向传播**

- **前向传播**:

- 输入: $(X \in \mathbb{R}^{B \times S \times D})$ (批次大小 B , 序列长度 S , 模型维度 D)。

- 投影: $(Q = X W_Q, K = X W_K, V = X W_V, W_Q, W_K, W_V \in \mathbb{R}^{D \times D})$ 。

- 多头拆分: $(Q_h, K_h, V_h \in \mathbb{R}^{B \times S \times D_h}), (H)$ 个头, $(D_h = D/H)$ 。

- 注意力分数: $(S_h = \frac{Q_h K_h^T}{\sqrt{D_h}} \in \mathbb{R}^{B \times S \times S}), (P_h = \text{Softmax}(S_h))$ 。

- 输出: $(A_h = P_h V_h \in \mathbb{R}^{B \times S \times D_h})$, 合并为 $(A \in \mathbb{R}^{B \times S \times D})$ 。

- **反向传播**:

- 输入梯度: $(\frac{\partial L}{\partial A_h} \in \mathbb{R}^{B \times S \times D_h})$ 。

- 梯度计算:

- $(\frac{\partial L}{\partial P_h} = \frac{\partial L}{\partial A_h} V_h^T \in \mathbb{R}^{B \times S \times S})$ 。

- $(\frac{\partial L}{\partial S_h} = P_h \cdot \left(\frac{\partial L}{\partial P_h} - \sum_j P_h[:, j, :] \frac{\partial L}{\partial P_h}[:, j, :] \right) \in \mathbb{R}^{B \times S \times S})$ 。

- $(\frac{\partial L}{\partial Q_h} = \frac{\partial L}{\partial S_h} K_h), (\frac{\partial L}{\partial K_h} = \frac{\partial L}{\partial S_h^T} Q_h), (\frac{\partial L}{\partial V_h} = P_h^T \frac{\partial L}{\partial A_h})$ 。

- $(\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q_h})$, 类似 (W_K, W_V) 。

- **HBM 读写问题**:

- 存储 $(S_h, P_h \in \mathbb{R}^{B \times H \times S \times S})$, 内存 $(O(BHS^2))$ 。

- 反向传播需读 (S_h, P_h, V_h) ($\sim 3BHS^2$ bytes) , 写 $(\frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h})$ ($\sim 2BHS^2$ bytes) , HBM 访问频繁 (例如, $(B=1, H=32, S=2048)$, $\sim 1.5GB$ 读写) 。

2. **FlashAttention v1 优化**

- **核心优化**:

- **块状计算**: 将 (Q, K, V) 分块 (块大小 $(S_r \parallel S)$) , 逐块计算梯度, 内存从 $(O(S^2))$ 降到 $(O(S))$.

- **在线 Softmax 梯度**: 在块内计算 Softmax 梯度, 避免存储 $(\frac{\partial L}{\partial S_h})$.

- **内核融合**: 融合梯度计算步骤, 数据保留在 SRAM, 减少 HBM 读写。

- **反向传播优势**:

- 内存: 仅存储块数据 (如 $(S_{ij} \in \mathbb{R}^{B \times S_r \times S_r})$) , $\sim$ 几十 KB。

- HBM 访问: 读 (Q_i, K_j, V_j, P_{ij}) , 写梯度, 远低于标准注意力。

二、具体示例: FlashAttention v1 反向传播

1. **场景**

- **输入**:

- $(X \in \mathbb{R}^{1 \times 8 \times 4})$ ($(B=1, S=8, D=4)$) 。

- 头数: $(H=2, D_h = 4/2 = 2)$.

- **参数**:

- $(W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4})$.

- **块大小**:

- 序列分块: $(S_r=4)$, 2 块 (0-3, 4-7) 。

- **硬件**: NVIDIA A100, HBM $\sim 40GB$, SRAM $\sim 10MB/SM$.

- **输入矩阵**:

```
\[
X = \begin{bmatrix}
1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 \\
1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 \\
1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 \\
1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0
\end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}
\]
```

- **权重** (简化, 单位矩阵) :

```
\[
W_Q = W_K = W_V = \begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1
\end{bmatrix}
\]
```

```

0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1 \\
\end{bmatrix}
\]

```

- **输入梯度** (假设) :

```

\[\frac{\partial L}{\partial A_0} = \begin{bmatrix}
1 & 0 \\
0 & 1 \\
1 & 0 \\
0 & 1 \\
1 & 0 \\
0 & 1 \\
1 & 0 \\
0 & 1
\end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 2} \text{ (头 0)}
\]

```

2. **前向传播 (简述, 参考前述) **

- 投影: $(Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4})$.

- 头 0: $(Q_0, K_0, V_0 = X[:, :, 0:2])$.

- 块 1-1 $(Q_0[0:4], K_0[0:4], V_0[0:4])$:

```

\[\begin{aligned}
S_{11} &= \frac{Q_0[0:4] K_0[0:4]^T}{\sqrt{2}} = \begin{bmatrix}
1 & 0 & 1 & 0 \\
0 & 1 & 0 & 1 \\
1 & 0 & 1 & 0 \\
0 & 1 & 0 & 1
\end{bmatrix}, \quad P_{11} = \text{Softmax}(S_{11}) \approx \begin{bmatrix}
0.5 & 0 & 0.5 & 0 \\
0 & 0.5 & 0 & 0.5 \\
0.5 & 0 & 0.5 & 0 \\
0 & 0.5 & 0 & 0.5
\end{bmatrix} \\
\end{aligned}
\]

```

```

\[\begin{aligned}
A_{11} &= P_{11} V_0[0:4] \approx \begin{bmatrix}
1 & 0 \\
0 & 1 \\
1 & 0 \\
0 & 1
\end{bmatrix} \\
\end{aligned}
\]

```

- 块 1-2: 类似, $(A_{12} \approx \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix})$.

- 在线 Softmax: 合并 $(A_1[0:4] \approx \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix})$.

- 输出: $(A_0 \in \mathbb{R}^{1 \times 8 \times 2})$.

- ****分块****:

- 梯度: $(\frac{\partial L}{\partial A_0} \in \mathbb{R}^{1 \times 8 \times 2})$, 分块处理。

1. **块 1-1 $(\backslash(Q_0[0:4], K_0[0:4], V_0[0:4], \frac{\partial L}{\partial A_0[0:4]})$ **:

 $\setminus [$

$$\begin{bmatrix}$$

V

$$\end{bmatrix} \end{matrix} \backslash).$$
 $\vee$

$$\end{bmatrix}$$

 $\sqrt{}$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```

0 & 0 \\
0 & 0 \\
0 & 0 \\
0 & 0
\end{bmatrix}, \quad \frac{\partial L}{\partial K_0[0:4]} = \frac{\partial L}{\partial S_{11}}^T
Q_0[0:4] \approx \begin{bmatrix}
0 & 0 \\
0 & 0 \\
0 & 0 \\
0 & 0
\end{bmatrix}
\end{bmatrix}
\end{pre}

```

2. **块 1-2 $(Q_0[0:4], K_0[4:8], V_0[4:8], \frac{\partial L}{\partial A_0[0:4]})$ **:

- 类似计算, 得到 $(\frac{\partial L}{\partial P_{12}}, \frac{\partial L}{\partial S_{12}}, \frac{\partial L}{\partial V_0[4:8]})$.

3. **在线梯度合并**:

- 累积梯度: $(\frac{\partial L}{\partial V_0} = \sum_j P_{1j}^T \frac{\partial L}{\partial A_0})$.
- $(\frac{\partial L}{\partial Q_0[0:4]} = \sum_j \frac{\partial L}{\partial S_{1j}} K_j, (\frac{\partial L}{\partial K_j} = \sum_i \frac{\partial L}{\partial S_{ij}}^T Q_i))$.

4. **参数梯度**:

- $(\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q_0})$, 类似 (W_K, W_V) .
- **HBM 读写**:

- 读: $(Q_i, K_j, V_j, P_{ij}, \frac{\partial L}{\partial A_i}) \approx 5 \times 1 \times 4 \times 2 \times 2 = 40$ bytes/块.

- 写: $(\frac{\partial L}{\partial Q_i}, \frac{\partial L}{\partial K_j}, \frac{\partial L}{\partial V_j}) \approx 3 \times 16$ bytes/块.

- 总计: ~88 bytes/块, 2 块 ~176 bytes.

- **SRAM**:

- 存储: $(S_{ij}, P_{ij}, \frac{\partial L}{\partial S_{ij}}) \approx 3 \times 1 \times 4 \times 4 \times 2 = 96$ bytes, 适配 SRAM (~192KB/SM).

- **对比标准注意力**:

- 标准: 读 $(S_0, P_0, V_0) \approx 3 \times 128$ bytes, 写 $(\frac{\partial L}{\partial S_0}, \frac{\partial L}{\partial P_0}) \approx 2 \times 128$ bytes, 总计 ~640 bytes.

- FlashAttention: ~176 bytes, HBM 访问降到 ~1/4.

4. **流程图 (文本形式) **

- **标准注意力反向传播**:

...

$[\partial L / \partial A_h: 1 \times 8 \times 2] \rightarrow [\partial L / \partial P_h = \partial L / \partial A_h V_h^T] \rightarrow [\partial L / \partial S_h = P_h * (\partial L / \partial P_h - \Sigma)] \rightarrow [\partial L / \partial Q_h, \partial L / \partial K_h, \partial L / \partial V_h] \rightarrow [\partial L / \partial W_Q, \partial L / \partial W_K, \partial L / \partial W_V]$

[HBM 读: 32B]	[HBM 读: 160B, 写: 128B]	[HBM 读: 256B, 写: 128B]	[HBM 读: 256B, 写: 96B]	[HBM 写: 96B]

[内存: 32B]	[内存: 128B]	[内存: 128B]	[内存: 96B]	[输出]
-----------	------------	------------	-----------	------

...

- **FlashAttention 反向传播**:

$[\partial L / \partial A_i: 1 \times 4 \times 2] \rightarrow [\text{Block: } Q_i, K_j, V_j, P_{ij}] \rightarrow [\partial L / \partial P_{ij}, \partial L / \partial S_{ij}] \rightarrow [\partial L / \partial Q_i, \partial L / \partial K_j, \partial L / \partial V_j] \rightarrow [\partial L / \partial W_Q, \partial L / \partial W_K, \partial L / \partial W_V]$

[HBM 读: 16B]	[SRAM: 96B]	[SRAM: 96B]	[HBM 写: 48B]	[HBM 累积]
[内存: 16B]	[HBM 读: 40B/块]	[内存释放]	[输出]	

三、为什么 FlashAttention 减少 HBM 读写

1. **块状计算**:

- 分块 ($\setminus(S_r=4 \setminus)$) 将 $\setminus(S_h, P_h \in \mathbb{R}^{1 \times 8 \times 8} \setminus)$ 拆成 $\setminus(S_{ij}, P_{ij}) \in \mathbb{R}^{1 \times 4 \times 4} \setminus$, 内存从 128 bytes 降到 32 bytes, 适配 SRAM。

2. **在线 Softmax 梯度**:

- 直接计算 $\setminus(\frac{\partial L}{\partial S_{ij}} \setminus)$ 在 SRAM, 无需存储整个 $\setminus(\frac{\partial L}{\partial S_h} \setminus)$ 。

3. **内核融合**:

- 融合 $\setminus(\frac{\partial L}{\partial P_{ij}} \setminus), \setminus(\frac{\partial L}{\partial S_{ij}} \setminus), \setminus(\frac{\partial L}{\partial Q_i}, \frac{\partial L}{\partial K_j}, \frac{\partial L}{\partial V_j} \setminus)$ 到单个内核, 数据保留在 SRAM。

4. **HBM 访问**:

- 标准: 读写 ~ 640 bytes ($\setminus(S_h, P_h, V_h, \frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h} \setminus)$)。

- FlashAttention: 读写 ~ 176 bytes (块数据和梯度), HBM 访问降到 $\sim 1/4$ 。

4. **LLaMA-3.1-8B 示例**

- **场景**:

- $\setminus(B=16, S=128, D=4096, H=32, D_h=128 \setminus), \setminus(S_r=64 \setminus)$ 。

- **标准注意力**:

- HBM: 读 $\setminus(S_h, P_h, V_h \approx 24 \setminus)$ GB, 写 $\setminus(\frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h} \approx 16 \setminus)$ GB, 总计 ~ 40 GB, 延迟 ~ 20 ms (2TB/s)。

- **FlashAttention**:

- SRAM: $\setminus(S_{ij}, P_{ij}, \frac{\partial L}{\partial S_{ij}} \approx 3 \times 16 \times 32 \times 64 \times 64 \times 2 \approx 6 \setminus)$ MB。

- HBM: 读 ~ 3 MB/块, 写 ~ 2 MB/块, 总计 ~ 10 MB, 延迟 $\sim 5\mu$ s。

- 内存: 降到 $\sim 0.03\%$ (6MB vs 16GB), 速度提升 ~ 2 x。

四、注意事项

- ****SRAM 限制****: 块大小 $\backslash(S_r\backslash)$ 受 SRAM 容量限制 (A100 ~10MB) 。
- ****硬件支持****: 需 GPU 快速 SRAM 和 CUDA 优化。
- ****Grok 3**** (<https://x.ai/api>): Grok 3 长上下文 (>100k tokens) 需 FlashAttention 优化反向传播内存。

五、总结

- ****FlashAttention v1 反向传播****:
 - 块状计算和在线 Softmax 梯度将内存从 $\backslash(O(S^2)\backslash)$ 降到 $\backslash(O(S)\backslash)$ 。
 - 内核融合在 SRAM 计算, HBM 读写从 ~640 bytes 降到 ~176 bytes。
- ****示例****: $\backslash(S=8, S_r=4\backslash)$, FlashAttention 处理块梯度, 内存降到 ~96 bytes, HBM 访问降到 ~1/4。
- ****效果****: 内存和速度优化, 适合长序列训练。

如果你需要 Chart.js 流程图、更复杂示例或与 FSDP/张量并行的对比, 请告诉我, 我可以进一步定制!

六、代码示例

以下是 FlashAttention v1 的简化 PyTorch 伪代码, 展示反向传播逻辑:

```
python
import torch

def flash_attention_backward(Q, K, V, P, dA, Sr=4):
    B, S, D = Q.shape
    H = 2 # 简化, 2 个头
    Dh = D // H
    dQ, dK, dV = torch.zeros_like(Q), torch.zeros_like(K), torch.zeros_like(V)

    # 分块循环
    for i in range(0, S, Sr):
        Qi = Q[:, i:i+Sr, :] # B x Sr x D
        dAi = dA[:, i:i+Sr, :] # B x Sr x D
        for j in range(0, S, Sr):
            Kj, Vj = K[:, j:j+Sr, :], V[:, j:j+Sr, :]
            Pij = P[:, i:i+Sr, j:j+Sr] # 前向保存的 P
            # 梯度计算
            dPij = dAi @ Vj.transpose(-2, -1) #  $\partial L / \partial P = \partial L / \partial A V^T$ 
            dSij = Pij * (dPij - (Pij * dPij).sum(dim=-1, keepdim=True)) #  $\partial L / \partial S$ 
            dV[:, j:j+Sr, :] += Pij.transpose(-2, -1) @ dAi #  $\partial L / \partial V$ 
            dQ[:, i:i+Sr, :] += dSij @ Kj #  $\partial L / \partial Q$ 
            dK[:, j:j+Sr, :] += dSij.transpose(-2, -1) @ Qi #  $\partial L / \partial K$ 
```

```

# 参数梯度
dWq = X.transpose(-2, -1) @ dQ
dWk = X.transpose(-2, -1) @ dK
dWv = X.transpose(-2, -1) @ dV
return dWq, dWk, dWv

# 示例数据
X = torch.tensor([[[[1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0]]], dtype=torch.float32)
Q, K, V = X, X, X # 简化
P = torch.ones(1, 8, 8) * 0.25 # 假设前向 Softmax 输出
dA = torch.ones(1, 8, 4) # 输入梯度
dWq, dWk, dWv = flash_attention_backward(Q, K, V, P, dA, Sr=4)
'''

---
```

如果你需要更详细的数学推导、真实 LLaMA-8B 示例或与标准注意力的对比，请告诉我，我可以进一步扩展！

感谢你的提问！你要求通过一个具体例子详细说明 **FlashAttention v1** 在反向传播（backward propagation）中的工作原理，包括矩阵、数据、流程图和端到端示例。FlashAttention v1 是一种高效的注意力机制实现，通过块状计算、在线 Softmax 和内核融合优化了 Transformer 多头自注意力（MHSA）的内存和计算效率，特别是在反向传播中减少了高带宽内存（HBM）读写。我将通过一个简化示例，结合矩阵、数字和文本形式的流程图，清晰展示 FlashAttention v1 的反向传播流程，解释其优化机制，并与标准注意力机制对比。参考资料包括 FlashAttention 论文（Dao et al., 2022）、PyTorch 实现和 GPU 内存架构文献。

一、背景：FlashAttention v1 与反向传播

1. 标准注意力反向传播

- 前向传播：

- 输入： $X \in \mathbb{R}^{B \times S \times D}$ （批次大小 B ，序列长度 S ，模型维度 D ）。
- 投影： $Q = XW_Q, K = XW_K, V = XW_V, W_Q, W_K, W_V \in \mathbb{R}^{D \times D}$ 。
- 多头拆分： $Q_h, K_h, V_h \in \mathbb{R}^{B \times S \times D_h}$ ， H 个头， $D_h = D/H$ 。
- 注意力分数： $S_h = \frac{Q_h K_h^T}{\sqrt{D_h}} \in \mathbb{R}^{B \times S \times S}$ ， $P_h = \text{Softmax}(S_h)$ 。
- 输出： $A_h = P_h V_h \in \mathbb{R}^{B \times S \times D_h}$ ，合并为 $A \in \mathbb{R}^{B \times S \times D}$ 。

- 反向传播：

- 输入梯度： $\frac{\partial L}{\partial A_h} \in \mathbb{R}^{B \times S \times D_h}$ 。
- 梯度计算：
 - $\frac{\partial L}{\partial P_h} = \frac{\partial L}{\partial A_h} V_h^T \in \mathbb{R}^{B \times S \times S}$ 。
 - $\frac{\partial L}{\partial S_h} = P_h \cdot \left(\frac{\partial L}{\partial P_h} - \sum_j P_h[:, j, :] \frac{\partial L}{\partial P_h}[:, j, :] \right) \in \mathbb{R}^{B \times S \times S}$ 。
 - $\frac{\partial L}{\partial Q_h} = \frac{\partial L}{\partial S_h} K_h, \frac{\partial L}{\partial K_h} = \frac{\partial L}{\partial S_h}^T Q_h, \frac{\partial L}{\partial V_h} = P_h^T \frac{\partial L}{\partial A_h}$ 。

by

- 反向传播:

- 输入梯度: $\frac{\partial L}{\partial A_h} \in \mathbb{R}^{B \times S \times D_h}$.
- 梯度计算:
 - $\frac{\partial L}{\partial P_h} = \frac{\partial L}{\partial A_h} V_h^T \in \mathbb{R}^{B \times S \times S}$.
 - $\frac{\partial L}{\partial S_h} = P_h \cdot \left(\frac{\partial L}{\partial P_h} - \sum_j P_h[:, j, :] \frac{\partial L}{\partial P_h}[:, j, :] \right) \in \mathbb{R}^{B \times S \times S}$.
 - $\frac{\partial L}{\partial Q_h} = \frac{\partial L}{\partial S_h} K_h$, $\frac{\partial L}{\partial K_h} = \frac{\partial L}{\partial S_h}^T Q_h$, $\frac{\partial L}{\partial V_h} = P_h^T \frac{\partial L}{\partial A_h}$.
 - $\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q_h}$, 类似 W_K, W_V .

- HBM 读写问题:

- 存储 $S_h, P_h \in \mathbb{R}^{B \times H \times S \times S}$, 内存 $O(BHS^2)$.
- 反向传播需读 S_h, P_h, V_h ($\sim 3BHS^2$ bytes), 写 $\frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h}$ ($\sim 2BHS^2$ bytes), HBM 访问频繁 (例如, $B = 1, H = 32, S = 2048$, $\sim 1.5\text{GB}$ 读写)。

2. FlashAttention v1 优化

- 核心优化:

- 块状计算: 将 Q, K, V 分块 (块大小 $S_r \ll S$), 逐块计算梯度, 内存从 $O(S^2)$ 降到 $O(S)$.
- 在线 Softmax 梯度: 在块内计算 Softmax 梯度, 避免存储 $\frac{\partial L}{\partial S_h}$.
- 内核融合: 融合梯度计算步骤, 数据保留在 SRAM, 减少 HBM 读写。

- 反向传播优势:

- 内存: 仅存储块数据 (如 $S_{ij} \in \mathbb{R}^{B \times S_r \times S_r}$), $\sim$ 几十 KB。
- HBM 访问: 读 Q_i, K_j, V_j, P_{ij} , 写梯度, 远低于标准注意力。

反向传播优势：

- 内存：仅存储块数据（如 $S_{ij} \in \mathbb{R}^{B \times S_r \times S_r}$ ），~几十 KB。
- HBM 访问：读 Q_i, K_j, V_j, P_{ij} ，写梯度，远低于标准注意力。

、具体示例：FlashAttention v1 反向传播

场景

输入：

- $X \in \mathbb{R}^{1 \times 8 \times 4}$ ($B = 1, S = 8, D = 4$) 。
- 头数： $H = 2, D_h = 4/2 = 2$ 。

参数：

- $W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4}$ 。

块大小：

- 序列分块： $S_r = 4$ ，2 块（0-3, 4-7）。

硬件： NVIDIA A100， HBM ~40GB， SRAM ~10MB/SM。

输入矩阵：

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}$$

- 输入矩阵：

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}$$

- 权重（简化，单位矩阵）：

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 输入梯度（假设）：

$$\frac{\partial L}{\partial A_0} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 2} (\text{头 } 0)$$

2. 前向传播（简述，参考前述）

- 投影： $Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4}$.
- 头 0： $Q_0, K_0, V_0 = X[:, :, 0 : 2]$.

2. 前向传播 (简述, 参考前述)

- 投影: $Q = K = V = X \in \mathbb{R}^{1 \times 8 \times 4}$.
- 头 0: $Q_0, K_0, V_0 = X[:, :, 0 : 2]$.
- 块 1-1 ($Q_0[0 : 4], K_0[0 : 4], V_0[0 : 4]$):

$$S_{11} = \frac{Q_0[0 : 4]K_0[0 : 4]^T}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{11} = \text{Softmax}(S_{11}) \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{11} = P_{11}V_0[0 : 4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 块 1-2: 类似, $A_{12} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$.

- 在线 Softmax: 合并 $A_1[0 : 4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$.

- 输出: $A_0 \in \mathbb{R}^{1 \times 8 \times 2}$.

3. 反向传播: FlashAttention

- 分块:

- $Q_0, K_0, V_0 \in \mathbb{R}^{1 \times 8 \times 2}$, 分 2 块 ($S_r = 4$) 。



- $\frac{\partial L}{\partial V_0[0:4]}$:

$$\frac{\partial L}{\partial V_0[0:4]} = P_{11}^T \frac{\partial L}{\partial A_0[0:4]} \approx \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

- $\frac{\partial L}{\partial Q_0[0:4]}, \frac{\partial L}{\partial K_0[0:4]}$:

$$\frac{\partial L}{\partial Q_0[0:4]} = \frac{\partial L}{\partial S_{11}} K_0[0:4] \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad \frac{\partial L}{\partial K_0[0:4]} = \frac{\partial L}{\partial S_{11}}^T Q_0[0:4] \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

2. 块 1-2 ($Q_0[0:4], K_0[4:8], V_0[4:8], \frac{\partial L}{\partial A_0[0:4]}$):

- 类似计算, 得到 $\frac{\partial L}{\partial P_{12}}, \frac{\partial L}{\partial S_{12}}, \frac{\partial L}{\partial V_0[4:8]}$.

3. 在线梯度合并:

- 累积梯度: $\frac{\partial L}{\partial V_0} = \sum_j P_{1j}^T \frac{\partial L}{\partial A_0}$.
- $\frac{\partial L}{\partial Q_0[0:4]} = \sum_j \frac{\partial L}{\partial S_{1j}} K_j, \frac{\partial L}{\partial K_j} = \sum_i \frac{\partial L}{\partial S_{ij}}^T Q_i$.

4. 参数梯度:

- $\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q_0}$, 类似 W_K, W_V .

• HBM 读写:

- 读: $Q_i, K_j, V_j, P_{ij}, \frac{\partial L}{\partial A_i} \approx 5 \times 1 \times 4 \times 2 \times 2 = 40$ bytes/块.
- 写: $\frac{\partial L}{\partial Q_i}, \frac{\partial L}{\partial K_j}, \frac{\partial L}{\partial V_j} \approx 3 \times 16$ bytes/块.
- 总计: ~88 bytes/块, 2 块 ~176 bytes.

• SRAM:

- **SRAM:**
 - 存储: $S_{ij}, P_{ij}, \frac{\partial L}{\partial S_{ij}} \approx 3 \times 1 \times 4 \times 4 \times 2 = 96$ bytes, 适配 SRAM (~192KB/SM).
- **对比标准注意力:**
 - 标准: 读 $S_0, P_0, V_0 \approx 3 \times 128$ bytes, 写 $\frac{\partial L}{\partial S_0}, \frac{\partial L}{\partial P_0} \approx 2 \times 128$ bytes, 总计 ~640 bytes.
 - FlashAttention: ~176 bytes, HBM 访问降到 ~1/4.

4. 流程图 (文本形式)

- **标准注意力反向传播:**

```

text
[∂L/∂A_h: 1×8×2] → [∂L/∂P_h = ∂L/∂A_h V_h^T] → [∂L/∂S_h = P_h * (∂L/∂P_h - Σ)] → [∂L/∂Q_i, ∂L/∂K_j, ∂L/∂V_j]
|           |           |           |
[HBM 读: 32B]   [HBM 读: 160B, 写: 128B]   [HBM 读: 256B, 写: 128B]   [HBM 读: 256B, 写: 128B]
[内存: 32B]     [内存: 128B]                 [内存: 128B]                 [内存: 96B]

```

- **FlashAttention 反向传播:**

```

text
[∂L/∂A_i: 1×4×2] → [Block: Q_i, K_j, V_j, P_ij] → [∂L/∂P_ij, ∂L/∂S_ij] → [∂L/∂Q_i, ∂L/∂K_j, ∂L/∂V_j]
|           |           |           |
[HBM 读: 16B]   [SRAM: 96B]           [SRAM: 96B]           [HBM 写: 48B]
[内存: 16B]     [HBM 读: 40B/块]       [内存释放]             [输出]

```

三、为什么 FlashAttention 减少 HBM 读写

三、为什么 FlashAttention 减少 HBM 读写

1. 块状计算：

- 分块 ($S_r = 4$) 将 $S_h, P_h \in \mathbb{R}^{1 \times 8 \times 8}$ 拆成 $S_{ij}, P_{ij} \in \mathbb{R}^{1 \times 4 \times 4}$ ，内存从 128 bytes 降到 32 bytes，适配 SRAM。

2. 在线 Softmax 梯度：

- 直接计算 $\frac{\partial L}{\partial S_{ij}}$ 在 SRAM，无需存储整个 $\frac{\partial L}{\partial S_h}$ 。

3. 内核融合：

- 融合 $\frac{\partial L}{\partial P_{ij}}, \frac{\partial L}{\partial S_{ij}}, \frac{\partial L}{\partial Q_i}, \frac{\partial L}{\partial K_j}, \frac{\partial L}{\partial V_j}$ 到单个内核，数据保留在 SRAM。

4. HBM 访问：

- 标准：读写 ~ 640 bytes ($S_h, P_h, V_h, \frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h}$)。
- FlashAttention：读写 ~ 176 bytes (块数据和梯度)，HBM 访问降到 $\sim 1/4$ 。

4. LLaMA-3.1-8B 示例

• 场景：

- $B = 16, S = 128, D = 4096, H = 32, D_h = 128, S_r = 64$ 。

• 标准注意力：

- HBM：读 $S_h, P_h, V_h \approx 24\text{GB}$ ，写 $\frac{\partial L}{\partial S_h}, \frac{\partial L}{\partial P_h} \approx 16\text{GB}$ ，总计 $\sim 40\text{GB}$ ，延迟 $\sim 20\text{ms}$ (2TB/s)。

• FlashAttention：

- SRAM： $S_{ij}, P_{ij}, \frac{\partial L}{\partial S_{ij}} \approx 3 \times 16 \times 32 \times 64 \times 64 \times 2 \approx 6\text{MB}$ 。
- HBM：读 $\sim 3\text{MB/块}$ ，写 $\sim 2\text{MB/块}$ ，总计 $\sim 10\text{MB}$ ，延迟 $\sim 5\mu\text{s}$ 。

- **FlashAttention:**

- SRAM: $S_{ij}, P_{ij}, \frac{\partial L}{\partial S_{ij}} \approx 3 \times 16 \times 32 \times 64 \times 64 \times 2 \approx 6\text{MB}$.
- HBM: 读 ~3MB/块, 写 ~2MB/块, 总计 ~10MB, 延迟 ~5 μs .
- 内存: 降到 ~0.03% (6MB vs 16GB), 速度提升 ~2x.

四、注意事项

- **SRAM 限制:** 块大小 S_r 受 SRAM 容量限制 (A100 ~10MB)。
- **硬件支持:** 需 GPU 快速 SRAM 和 CUDA 优化。
- **Grok 3:** Grok 3 长上下文 (>100k tokens) 需 FlashAttention 优化反向传播内存。

五、总结

- **FlashAttention v1 反向传播:**
 - 块状计算和在线 Softmax 梯度将内存从 $O(S^2)$ 降到 $O(S)$.
 - 内核融合在 SRAM 计算, HBM 读写从 ~640 bytes 降到 ~176 bytes。
- **示例:** $S = 8, S_r = 4$, FlashAttention 处理块梯度, 内存降到 ~96 bytes, HBM 访问降到 ~1/4。
- **效果:** 内存和速度优化, 适合长序列训练。

如果你需要 Chart.js 流程图、更复杂示例或与 FSDP/张量并行的对比, 请告诉我, 我可以进一步定制!

六、代码示例

以下是 FlashAttention v1 的简化 PyTorch 伪代码, 展示反向传播逻辑:

//-----

//-----

Code:

Fav1 backward propagation

import torch

def flash_attention_backward(Q, K, V, P, dA, Sr=4):

B, S, D = Q.shape

H = 2 # 简化, 2 个头

Dh = D // H

dQ, dK, dV = torch.zeros_like(Q), torch.zeros_like(K), torch.zeros_like(V)

分块循环

for i in range(0, S, Sr):

Qi = Q[:, i:i+Sr, :] # B×Sr×D

dAi = dA[:, i:i+Sr, :] # B×Sr×D

for j in range(0, S, Sr):

Kj, Vj = K[:, j:j+Sr, :], V[:, j:j+Sr, :]

Pij = P[:, i:i+Sr, j:j+Sr] # 前向保存的 P

梯度计算

dPij = dAi @ Vj.transpose(-2, -1) # $\partial L / \partial P = \partial L / \partial A V^T$

dSij = Pij * (dPij - (Pij * dPij).sum(dim=-1, keepdim=True)) # $\partial L / \partial S$

dV[:, j:j+Sr, :] += Pij.transpose(-2, -1) @ dAi # $\partial L / \partial V$

dQ[:, i:i+Sr, :] += dSij @ Kj # $\partial L / \partial Q$

dK[:, j:j+Sr, :] += dSij.transpose(-2, -1) @ Qi # $\partial L / \partial K$

参数梯度

dWq = X.transpose(-2, -1) @ dQ

dWk = X.transpose(-2, -1) @ dK

dWv = X.transpose(-2, -1) @ dV

return dWq, dWk, dWv

示例数据

X = torch.tensor([[[[1, 0, 0, 1],

[0, 1, 1, 0],

[1, 0, 0, 1],

[0, 1, 1, 0],

[1, 0, 0, 1],

[0, 1, 1, 0],

[1, 0, 0, 1],

```

        [0, 1, 1, 0]], dtype=torch.float32)
Q, K, V = X, X, X # 简化
P = torch.ones(1, 8, 8) * 0.25 # 假设前向 Softmax 输出
dA = torch.ones(1, 8, 4) # 输入梯度
dWq, dWk, dWv = flash_attention_backward(Q, K, V, P, dA, Sr=4)

```

```

//-----
//-----
//-----

//-----
// fa v2
//-----

```

二、具体示例：FlashAttention v2 前向传播，似乎和flash attention v1的例子是一样的，有错误吗？请更正

感谢你的细致观察！你指出我在回答中提供的 **FlashAttention v2 前向传播** 示例与 **FlashAttention v1** 示例看起来相同，并询问是否有错误。经检查，我确认示例在数据和矩阵计算上确实未充分体现 FlashAttention v2 相较于 v1 的具体优化（例如序列长度并行和线程块工作分配的改进），虽然核心分块和在线 Softmax 逻辑相似，但 v2 的实现细节（如并行策略）未在示例中明确区分，导致表述不够清晰。这是我的疏忽，我将更正并提供一个更准确的 FlashAttention v2 前向传播示例，突出其与 v1 的差异，包含矩阵、数据、流程图和端到端例子，并确保与 v1 示例区分开。以下是更正后的回答，参考资料包括 FlashAttention 论文（Dao et al., 2022）、FlashAttention-2 论文（Dao, 2023）、PyTorch 实现和 GPU 内存架构文献。

一、澄清与更正

1. 问题分析

- 原示例问题：

- 在之前的回答中，FlashAttention v2 的示例使用了与 v1 相同的输入矩阵、块大小和计算流程（ $S=8, S_r=4, B=1, H=2, D=4$ ），未明确体现 v2 的优化点（如序列长度并行、warp 级工作分配）。
- 虽然分块和在线 Softmax 逻辑在 v1 和 v2 中相似，v2 的线程块分配（sequence parallelism）和工作划分（4 warps 分片 Q ，共享 K, V ）未在示例中清晰展示，导致示例未能突出 v2 的性能提升（ $\sim 2\times$ FLOPS）。
- 流程图和 HBM 读写量相同，未反映 v2 减少非矩阵乘法（non-matmul）开销和共享内存通信的优势。

- 更正目标：

- 提供一个新示例，保持数据一致（便于对比），但明确展示 v2 的序列长度并行和线程块优化。
- 更新流程图，突出线程块并行和 warp 分配。
- 量化 v2 的性能提升（HBM 访问、FLOPS）。

2. FlashAttention v1 vs. v2 关键差异

- FlashAttention v1：

- **并行策略**：仅在批次（ B ）和头（ H ）维度并行，线程块数 = $B \times H$ 。
- **工作分配**：每个线程块处理一个头，内部使用“split-K”分片 K, V ，需 warp 间共享内存通信，增加开销。
- **性能**：A100 上 ~ 124 TFLOPS (FP16)，长序列时 SM 占用率低（ $\sim 25\text{-}40\%$ 峰值）。

- FlashAttention v2：

- **序列长度并行**：新增序列长度维度并行，线程块数 = $B \times H \times \lceil S/S_r \rceil$ ，适合长序列。

- **工作分配**：每个线程块处理 $(Q_i \in \mathbb{R}^{B \times S_r \times D_h})$ 的一块 ($Q_i \in \mathbb{R}^{B \times S_r \times D_h}$)，4 warps 分片 (Q_i) 的行，共享 (K_j, V_j) ，消除 warp 间通信。
- **性能**：A100 上 ~230 TFLOPS，接近 GEMM 峰值 (~312 TFLOPS)，减少非矩阵乘法 (Softmax、归一化) 开销。

二、更正后的 FlashAttention v2 前向传播示例

1. 场景

- **输入**：
 - $(X \in \mathbb{R}^{1 \times 8 \times 4})$ ($B=1, S=8, D=4$)。
 - 头数：($H=2, D_h = 4/2 = 2$)。
- **参数**：
 - $(W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4})$ 。
- **块大小**：
 - 序列分块：($S_r=4$)，2 块 (0-3, 4-7)。
 - Warp 分片：每个线程块内， $(Q_i \in \mathbb{R}^{1 \times 4 \times 2})$ 分给 4 warps，每个 warp 处理 1 行 ($1 \times 1 \times 2$)。
- **硬件**：
 - NVIDIA A100，HBM ~40GB (2TB/s)，SRAM ~128KB/SM (108 SMs, ~19TB/s)。
 - 假设 1 SM 处理 1 线程块，4 warps (128 线程) / 块。
- **输入矩阵** (与 v1 示例一致)：

$$\begin{bmatrix} X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4} \end{bmatrix}$$
- **权重** (简化，单位矩阵)：

$$\begin{bmatrix} W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{bmatrix}$$

2. FlashAttention v2 前向传播

- ****分块与并行****:
 - $(Q_0, K_0, V_0 = X[:, :, 0:2] \in \mathbb{R}^{1 \times 8 \times 2})$ (头 0)。
 - 序列分块: $(Q_0[0:4], Q_0[4:8])$, (K_0, V_0) 同理。
 - ****线程块分配****:
 - 线程块数 = $(B \times H \times \lceil S/S_r \rceil = 1 \times 2 \times \lceil 8/4 \rceil = 4)$ 。
 - 头 0: 2 块 $(Q_0[0:4], Q_0[4:8])$, 分配到 2 线程块)。
 - 每个线程块内: 4 warps, 每 warp 处理 (Q_i) 的 1 行 (例如, $(Q_0[0:4])$ 的 4 行分给 4 warps)。
 - ****Warp 分配****:
 - Warp 0: $(Q_0[0:1, 0:2] = [1, 0])$ 。
 - Warp 1: $(Q_0[1:2, 0:2] = [0, 1])$ 。
 - Warp 2: $(Q_0[2:3, 0:2] = [1, 0])$ 。
 - Warp 3: $(Q_0[3:4, 0:2] = [0, 1])$ 。
 - (K_j, V_j) 共享, 加载到共享内存。
- ****计算**** (以头 0, 线程块 1 处理 $(Q_0[0:4])$ 为例) :
 1. ****外循环 (K, V 块, $(j=0:4, 4:8)$) ****:
 - ****块 1-1 $(Q_0[0:4], K_0[0:4], V_0[0:4])$ ****:
 - 加载: $(Q_0[0:4], K_0[0:4], V_0[0:4]) \approx 24$ bytes (FP16)。
 - Warp 0 计算:

$$S_{11}[0, :] = \frac{Q_0[0:1] K_0[0:4]^T}{\sqrt{2}} = [1, 0, 1, 0]$$

$$P_{11}[0, :] = \text{softmax}([1, 0, 1, 0]) \approx [0.5, 0, 0.5, 0]$$

$$A_{11}[0, :] = P_{11}[0, :] V_0[0:4] \approx [1, 0]$$
 - Warp 1-3 类似, 得到:

$$S_{11} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{11} \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{11} = P_{11} V_0[0:4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

```

1 & 0 \\
0 & 1
\end{bmatrix}
\]
- **在线 Softmax** (每 warp) :
-  $(m_{11}[0] = \max(1, 0, 1, 0) \approx 1)$ ,  $(l_{11}[0] = \sum \exp([1, 0, 1, 0] - 1) \approx 2)$ .
- 合并:  $(A_{11})$  由各 warp 输出累积。
- **块 1-2  $(Q_0[0:4], K_0[4:8], V_0[4:8])$ ** :
- Warp 0:

$$S_{12}[0, :] = [1, 0, 1, 0], \quad P_{12}[0, :] \approx [0.5, 0, 0.5, 0]$$


$$A_{12}[0, :] \approx [1, 0]$$

- 合并:

$$S_{12} \approx \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{12} \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$


$$A_{12} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 在线 Softmax:
-  $(m_1 = \max(m_{11}, m_{12}) \approx 1)$ .
-  $(l_1 = l_{11} \exp(m_{11} - m_1) + l_{12} \exp(m_{12} - m_1) \approx 4)$ .
-  $(A_1[0:4] = \frac{\exp(m_{11} - m_1) A_{11} + \exp(m_{12} - m_1) A_{12}}{l_1} \approx$ 

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix})$$
.
2. **写回 HBM** :
- 写  $(A_1[0:4] \approx 16)$  bytes。

```

3. **线程块 2** $\backslash(Q_0[4:8])$:

- 类似处理, 4 warps 分片 $\backslash(Q_0[4:8])$, 得到 $\backslash(A_1[4:8])$.

- **HBM 读写**:

- 读: $\backslash(Q_i, K_j, V_j) \approx 3 \times 1 \times 4 \times 2 = 24$ bytes/块。

- 写: $\backslash(A_i) \approx 16$ bytes/块。

- 总计: 2 块, ~ 80 bytes (读 48 + 写 32)。

- **SRAM**:

- 存储: $\backslash(S_{ij}, P_{ij}) \in \mathbb{R}^{1 \times 4 \times 4} \approx 32$ bytes, 适配 SRAM ($\sim 128KB$)。

- Warp 分片: 每 warp 处理 $\backslash(S_{ij}[k, :]) \approx 8$ bytes, 共享 $\backslash(K_j, V_j) \approx 16$ bytes。

- **对比 v1**:

- v1: 线程块处理整个 $\backslash(Q_i)$, split-K 分片 $\backslash(K_j, V_j)$, 需 warp 间通信 ($\sim 10\%$ 共享内存开销)。

- v2: 4 warps 分片 $\backslash(Q_i)$, 共享 $\backslash(K_j, V_j)$, 无通信, FLOPS 提升 $\sim 2x$ 。

- **对比标准注意力**:

- 标准: 读写 ~ 416 bytes ($\backslash(S_h, P_h, V_h, A_h) \approx 3 \times 128 + 32$)。

- v2: ~ 80 bytes, HBM 访问降到 $\sim 1/5$ 。

3. **流程图 (文本形式)**

- **FlashAttention v1** (参考):

...

$[X: 1 \times 8 \times 4] \rightarrow [Q, K, V: 1 \times 8 \times 4] \rightarrow [\text{Block } Q[0:4], K[0:4], V[0:4]] \rightarrow [S_{11}, P_{11}, A_{11}] \rightarrow [A: 1 \times 8 \times 4]$

[HBM 读: 32B]	[HBM 读: 24B/块]	[SRAM: 32B, split-K]	[HBM 写: 16B/块]	[HBM: 32B]
[内存: 32B]	[内存: 24B/块]	[warp 通信: $\sim 8B$]	[输出]	[输出]

...

- **FlashAttention v2**:

...

$[X: 1 \times 8 \times 4] \rightarrow [Q, K, V: 1 \times 8 \times 4] \rightarrow [\text{Block } Q[0:4], K[0:4], V[0:4]] \rightarrow [S_{11}, P_{11}, A_{11}] \rightarrow [A: 1 \times 8 \times 4]$

[HBM 读: 32B]	[HBM 读: 24B/块]	[SRAM: 32B, 4 warps]	[HBM 写: 16B/块]	[HBM: 32B]
[内存: 32B]	[内存: 24B/块]	[Q 分片, 无 warp 通信]	[输出]	[输出]
[1 SM/块]	[4 warps 分 Q_i]	[FLOPS: ~ 230 TFLOPS]	[内存释放]	

...

三、FlashAttention v2 优化的体现

1. **序列长度并行**:

- v1: 线程块数 = $\backslash(B \times H = 1 \times 2 = 2)$, SM 占用率低 ($2/108 \approx 1.85\%$)。

- v2: 线程块数 = $\lfloor B \times H \times \lceil S/S_r \rceil \rfloor = 1 \times 2 \times 2 = 4$, SM 占用率提升 ($4/108 \approx 3.7\%$) 。

- 长序列 (如 $\lfloor S=2048, S_r=128 \rfloor$) , 线程块数 = $\lfloor 1 \times 32 \times 16 = 512 \rfloor$, SM 占用率 $\sim 100\%$ 。

2. **Warp 级工作分配**:

- v1: split-K 分片 $\lfloor (K_j, V_j) \rfloor$, warp 间共享内存同步 (~ 8 bytes 通信/块) 。

- v2: $\lfloor (Q_i) \rfloor$ 分 4 warps, $\lfloor (K_j, V_j) \rfloor$ 共享, 消除通信, SRAM 读写减少 $\sim 10\%$ 。

3. **FLOPS 提升**:

- v1: ~ 124 TFLOPS (A100, FP16) , 受非矩阵乘法 (Softmax) 限制。

- v2: ~ 230 TFLOPS, 优化 Softmax 调度, 接近 GEMM 峰值。

4. **HBM 访问**:

- 标准: ~ 416 bytes。

- v1/v2: ~ 80 bytes, v2 因更高并行度减少非矩阵乘法时间 ($\sim 0.1\mu s$ vs $\sim 0.2\mu s$) 。

4. **LLaMA-3.1-8B 示例**

- **场景**:

- $\lfloor (B=16, S=128, D=4096, H=32, D_h=128, S_r=64) \rfloor$ 。

- 线程块数: $\lfloor 16 \times 32 \times \lceil 128/64 \rceil \rfloor = 1024$, SM 占用率 $\sim 100\%$ (A100 108 SMs) 。

- **标准注意力**:

- HBM: $\lfloor (S_h, P_h) \approx 8 \rfloor$ GB, 读写 ~ 24 GB, 延迟 ~ 12 ms (2TB/s) 。

- **FlashAttention v2**:

- SRAM: $\lfloor (S_{ij}, P_{ij}) \approx 16 \times 32 \times 64 \times 64 \times 2 \approx 4 \rfloor$ MB。

- HBM: 读 ~ 3 MB/块, 写 ~ 0.5 MB/块, 总计 ~ 7 MB (2 块) , 延迟 $\sim 3.5\mu s$ 。

- 性能: ~ 230 TFLOPS, 速度提升 $\sim 9x$ (标准) , $\sim 2x$ (v1) 。

四、注意事项

- **SRAM 限制**: $\lfloor (S_r=64) \rfloor$ 适配 A100 SRAM (~ 128 KB/SM) , H100 (~ 228 KB/SM) 可更大。

- **硬件支持**: 需 A100/H100, Tensor Core 和 CUDA 优化。

- **Grok 3** (<https://x.ai/api>): 长上下文 ($>100k$ tokens) 依赖 v2 的高并行性。

五、总结

- **更正**: 原示例未体现 v2 的序列长度并行和 warp 分配优化, 导致与 v1 相似。

- **FlashAttention v2**:

- 序列长度并行: 线程块数增至 $\lfloor B \times H \times \lceil S/S_r \rceil \rfloor$, SM 占用率提升。

- Warp 分配: $\lfloor (Q_i) \rfloor$ 分片, $\lfloor (K_j, V_j) \rfloor$ 共享, 无 warp 通信, FLOPS ~ 230 TFLOPS。

- HBM: ~80 bytes (与 v1 相同) , 但非矩阵乘法时间减少。
- **示例**: $\backslash (S=8, S_r=4 \backslash)$, 4 warps 分片 $\backslash (Q_i \backslash)$, HBM 访问 ~80 bytes, SM 占用率翻倍。

六、代码示例

以下是 FlashAttention v2 前向传播的简化 PyTorch 伪代码, 突出序列长度并行:

```

python
import torch

def flash_attention_v2_forward(Q, K, V, Sr=4):
    B, S, D = Q.shape
    H = 2 # 简化, 2 个头
    Dh = D // H
    A = torch.zeros(B, S, D) # 输出
    l = torch.zeros(B, S) # Softmax 归一化因子
    m = torch.full((B, S), -float('inf')) # 最大值

    # 序列长度并行: 每个线程块处理 Q_i
    for i in range(0, S, Sr):
        Qi = Q[:, i:i+Sr, :] # B×Sr×D
        Ai = torch.zeros(B, Sr, D) # 块输出
        li = torch.zeros(B, Sr) # 块归一化因子
        mi = torch.full((B, Sr), -float('inf')) # 块最大值

        # 内循环: K, V 块, 共享内存
        for j in range(0, S, Sr):
            Kj, Vj = K[:, j:j+Sr, :], V[:, j:j+Sr, :]
            # 4 warps 分片 Q_i 的行
            for k in range(0, Sr): # 模拟 warp 分片
                Qik = Qi[:, k:k+1, :] # B×1×D
                Sijk = torch.matmul(Qik, Kj.transpose(-2, -1)) / (Dh ** 0.5) # B×1×Sr
                mijk = torch.max(Sijk, dim=-1)[0] # B×1
                Pijk = torch.exp(Sijk - mijk.unsqueeze(-1)) # B×1×Sr
                lijk = torch.sum(Pijk, dim=-1) # B×1
                Ai_new = torch.matmul(Pijk, Vj) # B×1×D
                mi_new = torch.max(mi[:, k], mijk)
                li_new = li[:, k] * torch.exp(mi[:, k] - mi_new) + lijk * torch.exp(mijk - mi_new)
                Ai[:, k:k+1, :] = li[:, k:k+1].unsqueeze(-1) * torch.exp(mi[:, k:k+1] -
                mi_new).unsqueeze(-1) * \
                    Ai[:, k:k+1, :] + torch.exp(mijk - mi_new).unsqueeze(-1) * Ai_new
                li[:, k], mi[:, k] = li_new, mi_new
            A[:, i:i+Sr, :] = Ai / li.unsqueeze(-1)
            l[:, i:i+Sr], m[:, i:i+Sr] = li, mi

    return A, l, m

```

```
# 示例数据
X = torch.tensor([[[[1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0],
                    [1, 0, 0, 1],
                    [0, 1, 1, 0]]]], dtype=torch.float32)
Q, K, V = X, X, X # 简化
A, l, m = flash_attention_v2_forward(Q, K, V, Sr=4)
'''
---
```

如果你需要 Chart.js 流程图、更复杂示例（如 LLaMA-8B）或与 v1/标准注意力的进一步对比，请告诉我，我可以进一步扩展！



Copy

感谢你的细致观察！你指出我在回答中提供的 **FlashAttention v2 前向传播** 示例与 **FlashAttention v1** 示例看起来相同，并询问是否有错误。经检查，我确认示例在数据和矩阵计算上确实未充分体现 FlashAttention v2 相较于 v1 的具体优化（例如序列长度并行和线程块工作分配的改进），虽然核心分块和在线 Softmax 逻辑相似，但 v2 的实现细节（如并行策略）未在示例中明确区分，导致表述不够清晰。这是我的疏忽，我将更正并提供一个更准确的 FlashAttention v2 前向传播示例，突出其与 v1 的差异，包含矩阵、数据、流程图和端到端例子，并确保与 v1 示例区分开。以下是更正后的回答，参考资料包括 FlashAttention 论文（Dao et al., 2022）、FlashAttention-2 论文（Dao, 2023）、PyTorch 实现和 GPU 内存架构文献。

一、澄清与更正

1. 问题分析

- **原示例问题：**
 - 在之前的回答中，FlashAttention v2 的示例使用了与 v1 相同的输入矩阵、块大小和计算流程（ $S = 8, S_r = 4, B = 1, H = 2, D = 4$ ），未明确体现 v2 的优化点（如序列长度并行、warp 级工作分配）。
 - 虽然分块和在线 Softmax 逻辑在 v1 和 v2 中相似，v2 的线程块分配（sequence parallelism）和工作划分（4 warps 分片 Q ，共享 K, V ）未在示例中清晰展示，导致示例未能突出 v2 的性能提升（~2x FLOPS）。
 - 流程图和 HBM 读写量相同，未反映 v2 减少非矩阵乘法（non-matmul）开销和共享内存通信的优势。
- **更正目标：**
 - 提供一个新示例，保持数据一致（便于对比），但明确展示 v2 的序列长度并行和线程块优化。
 - 更新流程图，突出线程块并行和 warp 分配。

- 更正目标：
 - 提供一个新示例，保持数据一致（便于对比），但明确展示 v2 的序列长度并行和线程块优化。
 - 更新流程图，突出线程块并行和 warp 分配。
 - 量化 v2 的性能提升（HBM 访问、FLOPS）。
- ## 2. FlashAttention v1 vs. v2 关键差异
- FlashAttention v1：
 - 并行策略：仅在批次 (B) 和头 (H) 维度并行，线程块数 $= B \times H$ 。
 - 工作分配：每个线程块处理一个头，内部使用“split-K”分片 K, V ，需 warp 间共享内存通信，增加开销。
 - 性能：A100 上 ~ 124 TFLOPS (FP16)，长序列时 SM 占用率低 ($\sim 25\text{-}40\%$ 峰值)。
 - FlashAttention v2：
 - 序列长度并行：新增序列长度维度并行，线程块数 $= B \times H \times \lceil S/S_r \rceil$ ，适合长序列。
 - 工作分配：每个线程块处理 Q 的一块 ($Q_i \in \mathbb{R}^{B \times S_r \times D_h}$)，4 warps 分片 Q_i 的行，共享 K_j, V_j ，消除 warp 间通信。
 - 性能：A100 上 ~ 230 TFLOPS，接近 GEMM 峰值 (~ 312 TFLOPS)，减少非矩阵乘法 (Softmax、归一化) 开销。

二、更正后的 FlashAttention v2 前向传播示例

1. 场景

- 输入：
 - $X \in \mathbb{R}^{1 \times 8 \times 4}$ ($B = 1, S = 8, D = 4$)。



二、更正后的 FlashAttention v2 前向传播示例

1. 场景

- 输入：
 - $X \in \mathbb{R}^{1 \times 8 \times 4}$ ($B = 1, S = 8, D = 4$) 。
 - 头数: $H = 2, D_h = 4/2 = 2$.
- 参数：
 - $W_Q, W_K, W_V \in \mathbb{R}^{4 \times 4}$.
- 块大小：
 - 序列分块: $S_r = 4$, 2 块 (0-3, 4-7) 。
 - Warp 分片: 每个线程块内, $Q_i \in \mathbb{R}^{1 \times 4 \times 2}$ 分给 4 warps, 每 warp 处理 1 行 ($1 \times 1 \times 2$) 。
- 硬件：
 - NVIDIA A100, HBM ~40GB (2TB/s) , SRAM ~128KB/SM (108 SMs, ~19TB/s) 。
 - 假设 1 SM 处理 1 线程块, 4 warps (128 线程) /块。
- 输入矩阵 (与 v1 示例一致) :

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1 \times 8 \times 4}$$



Copy

- 权重（简化，单位矩阵）：

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. FlashAttention v2 前向传播

- 分块与并行：
 - $Q_0, K_0, V_0 = X[:, :, 0 : 2] \in \mathbb{R}^{1 \times 8 \times 2}$ （头 0）。
 - 序列分块： $Q_0[0 : 4], Q_0[4 : 8]$ ， K_0, V_0 同理。
 - 线程块分配：
 - 线程块数 = $B \times H \times \lceil S/S_r \rceil = 1 \times 2 \times \lceil 8/4 \rceil = 4$ 。
 - 头 0： 2 块（ $Q_0[0 : 4], Q_0[4 : 8]$ ， 分配到 2 线程块）。
 - 每个线程块内： 4 warps， 每 warp 处理 Q_i 的 1 行（例如， $Q_0[0 : 4]$ 的 4 行分给 4 warps）。
 - Warp 分配：
 - Warp 0： $Q_0[0 : 1, 0 : 2] = [1, 0]$ 。
 - Warp 1： $Q_0[1 : 2, 0 : 2] = [0, 1]$ 。
 - Warp 2： $Q_0[2 : 3, 0 : 2] = [1, 0]$ 。
 - Warp 3： $Q_0[3 : 4, 0 : 2] = [0, 1]$ 。
 - K_j, V_j 共享， 加载到共享内存。
- 计算（以头 0， 线程块 1 处理 $Q_0[0 : 4]$ 为例）：
 1. 外循环（ K, V 块， $j = 0 : 4, 4 : 8$ ）：

- 计算 (以头 0, 线程块 1 处理 $Q_0[0:4]$ 为例) :

1. 外循环 (K, V 块, $j = 0:4, 4:8$) :

- 块 1-1 ($Q_0[0:4], K_0[0:4], V_0[0:4]$):
 - 加载: $Q_0[0:4], K_0[0:4], V_0[0:4] \approx 24$ bytes (FP16) 。
 - Warp 0 计算:

$$S_{11}[0, :] = \frac{Q_0[0:1]K_0[0:4]^T}{\sqrt{2}} = [1, 0, 1, 0]$$

$$P_{11}[0, :] = \text{Softmax}([1, 0, 1, 0]) \approx [0.5, 0, 0.5, 0]$$

$$A_{11}[0, :] = P_{11}[0, :]V_0[0:4] \approx [1, 0]$$

- Warp 1-3 类似, 得到:

$$S_{11} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{11} \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{11} = P_{11}V_0[0:4] \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 在线 Softmax (每 warp) :
 - $m_{11}[0] = \max(1, 0, 1, 0) \approx 1, l_{11}[0] = \sum \exp([1, 0, 1, 0] - 1) \approx 2.$
 - 合并: A_{11} 由各 warp 输出累积。

- 块 1-2 ($Q_0[0 : 4], K_0[4 : 8], V_0[4 : 8]$):

- Warp 0:

$$S_{12}[0, :] = [1, 0, 1, 0], \quad P_{12}[0, :] \approx [0.5, 0, 0.5, 0]$$

$$A_{12}[0, :] \approx [1, 0]$$

- 合并:

$$S_{12} \approx \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad P_{12} \approx \begin{bmatrix} 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \end{bmatrix}$$

$$A_{12} \approx \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 在线 Softmax:

- $m_1 = \max(m_{11}, m_{12}) \approx 1.$

- $l_1 = l_{11} \exp(m_{11} - m_1) + l_{12} \exp(m_{12} - m_1) \approx 4.$

-

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

2. 写回 HBM:

- 写 $A_1[0 : 4] \approx 16$ bytes。

3. 线程块 2 ($Q_0[4 : 8]$):

2. 写回 HBM:

- 写 $A_1[0 : 4] \approx 16$ bytes。

3. 线程块 2 ($Q_0[4 : 8]$):

- 类似处理, 4 warps 分片 $Q_0[4 : 8]$, 得到 $A_1[4 : 8]$.

• HBM 读写:

- 读: $Q_i, K_j, V_j \approx 3 \times 1 \times 4 \times 2 \times 2 = 24$ bytes/块。
- 写: $A_i \approx 16$ bytes/块。
- 总计: 2 块, ~ 80 bytes (读 48 + 写 32)。

• SRAM:

- 存储: $S_{ij}, P_{ij} \in \mathbb{R}^{1 \times 4 \times 4} \approx 32$ bytes, 适配 SRAM (~ 128 KB)。
- Warp 分片: 每 warp 处理 $S_{ij}[k, :] \approx 8$ bytes, 共享 $K_j, V_j \approx 16$ bytes。

• 对比 v1:

- v1: 线程块处理整个 Q_i , split-K 分片 K_j, V_j , 需 warp 间通信 ($\sim 10\%$ 共享内存开销)。
- v2: 4 warps 分片 Q_i , 共享 K_j, V_j , 无通信, FLOPS 提升 $\sim 2\times$ 。

• 对比标准注意力:

- 标准: 读写 ~ 416 bytes ($S_h, P_h, V_h, A_h \approx 3 \times 128 + 32$)。
- v2: ~ 80 bytes, HBM 访问降到 $\sim 1/5$ 。

3. 流程图 (文本形式)

- FlashAttention v1 (参考):

3. 流程图（文本形式）

- FlashAttention v1（参考）：

text

✖ Collapse ⇌ Wrap 📄 Copy

```
[X: 1×8×4] → [Q, K, V: 1×8×4] → [Block Q[0:4], K[0:4], V[0:4]] → [S_{11}, P_{11}, A_{11}]
|           |           |           |           |
[HBM 读: 32B] [HBM 读: 24B/块] [SRAM: 32B, split-K] [HBM 写: 16B/块] [HBM: 32B]
[内存: 32B]   [内存: 24B/块]   [warp 通信: ~8B]   [输出]           [输出]
```

- FlashAttention v2:

text

✖ Collapse ⇌ Wrap 📄 Copy

```
[X: 1×8×4] → [Q, K, V: 1×8×4] → [Block Q[0:4], K[0:4], V[0:4]] → [S_{11}, P_{11}, A_{11}]
|           |           |           |           |
[HBM 读: 32B] [HBM 读: 24B/块] [SRAM: 32B, 4 warps] [HBM 写: 16B/块] [HBM: 32B]
[内存: 32B]   [内存: 24B/块]   [Q 分片, 无 warp 通信] [输出]           [输出]
[1 SM/块]    [4 warps 分 Q_i] [FLOPS: ~230 TFLOPS]   [内存释放]
```

三、FlashAttention v2 优化的体现

1. 序列长度并行：

- v1: 线程块数 $= B \times H = 1 \times 2 = 2$, SM 占用率低 ($2/108 \approx 1.85\%$)。
- v2: 线程块数 $= B \times H \times \lceil S/S_r \rceil = 1 \times 2 \times 2 = 4$, SM 占用率提升 ($4/108 \approx 3.7\%$)。
- 长序列 (如 $S = 2048, S_r = 128$, 线程块数 $= 1 \times 32 \times 16 = 512$) , SM 占用率 $\sim 100\%$ 。

2. Warp 级工作分配：



2. Warp 级工作分配:

- v1: split-K 分片 K_j, V_j , warp 间共享内存同步 (~8 bytes 通信/块)。
- v2: Q_i 分 4 warps, K_j, V_j 共享, 消除通信, SRAM 读写减少 ~10%。

3. FLOPS 提升:

- v1: ~124 TFLOPS (A100, FP16), 受非矩阵乘法 (Softmax) 限制。
- v2: ~230 TFLOPS, 优化 Softmax 调度, 接近 GEMM 峰值。

4. HBM 访问:

- 标准: ~416 bytes。
- v1/v2: ~80 bytes, v2 因更高并行度减少非矩阵乘法时间 (~0.1 μ s vs ~0.2 μ s)。

4. LLaMA-3.1-8B 示例

• 场景:

- $B = 16, S = 128, D = 4096, H = 32, D_h = 128, S_r = 64$ 。
- 线程块数: $16 \times 32 \times \lceil 128/64 \rceil = 1024$, SM 占用率 ~100% (A100 108 SMs)

• 标准注意力:

- HBM: $S_h, P_h \approx 8\text{GB}$, 读写 ~24GB, 延迟 ~12ms (2TB/s)。

• FlashAttention v2:

- SRAM: $S_{ij}, P_{ij} \approx 16 \times 32 \times 64 \times 64 \times 2 \approx 4\text{MB}$ 。
- HBM: 读 ~3MB/块, 写 ~0.5MB/块, 总计 ~7MB (2 块), 延迟 ~3.5 μ s。
- 性能: ~230 TFLOPS, 速度提升 ~9x (标准), ~2x (v1)。

- 性能：~230 TFLOPS，速度提升 ~9x（标准），~2x（v1）。

四、注意事项

- SRAM 限制：** $S_r = 64$ 适配 A100 SRAM（~128KB/SM），H100（~228KB/SM）可更大。
- 硬件支持：**需 A100/H100，Tensor Core 和 CUDA 优化。
- Grok 3：**长上下文（>100k tokens）依赖 v2 的高并行性。

五、总结

- 更正：**原示例未体现 v2 的序列长度并行和 warp 分配优化，导致与 v1 相似。
- FlashAttention v2：**
 - 序列长度并行：线程块数增至 $B \times H \times \lceil S/S_r \rceil$ ，SM 占用率提升。
 - Warp 分配： Q_i 分片， K_j, V_j 共享，无 warp 通信，FLOPS ~230 TFLOPS。
 - HBM：~80 bytes（与 v1 相同），但非矩阵乘法时间减少。
- 示例：** $S = 8, S_r = 4$ ，4 warps 分片 Q_i ，HBM 访问 ~80 bytes，SM 占用率翻倍。

六、代码示例

以下是 FlashAttention v2 前向传播的简化 PyTorch 伪代码，突出序列长度并行：

 flash_attention_v2_forward.py
python

[Edit in files](#) • [Show inline](#)

//-----

//-----

//-----

//-----

// fa v2, 反向传播

//-----

请用一个具体例子说明flash attention v2, 反向propagation, 有矩阵, 数据, 流程图, 例子

我将用一个具体的例子来详细解释 **Flash Attention v2** 和 **反向传播 (Backpropagation)**, 并结合矩阵、数据和流程图来帮助理解。以下是一个简化但具体的例子, 假设我们在一个小型的 Transformer 模型中应用 Flash Attention v2, 并通过反向传播来优化参数。

背景

在 Transformer 模型中, **自注意力机制 (Self-Attention)** 是核心组件, 但传统注意力机制在计算和内存使用上效率较低。**Flash Attention v2** 是一种优化方法, 通过减少内存访问和利用 GPU 硬件特性来加速计算。**反向传播** 是深度学习中用于优化模型参数的算法, 通过计算损失函数对参数的梯度来更新权重。

我们将通过一个小型的注意力计算和反向传播的例子, 展示这两个过程。

例子: 小型自注意力计算与反向传播

场景

假设我们有一个小型输入序列, 包含 3 个 token (例如, 一个句子中的 3 个单词), 每个 token 的嵌入向量维度为 4。我们将计算自注意力并进行反向传播优化。

- **输入数据**: 矩阵 (X) 表示输入嵌入, 形状为 $(3, 4)$ (3 个 token, 每个 token 4 维)。
- **查询、键、值矩阵**: (W_Q, W_K, W_V) 是权重矩阵, 形状为 $(4, 4)$ 。
- **输出**: 计算注意力输出, 并通过一个简单的损失函数进行反向传播。

输入数据

假设输入嵌入矩阵 (X) 为:

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

```
\end{bmatrix}
\]
```

- 行表示 token，列表示嵌入维度。

权重矩阵（简化起见，假设初始化为简单值）：

```
\[
W_Q = W_K = W_V = \begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1
\end{bmatrix}
\]
```

（实际中这些矩阵会通过训练学习到更复杂的值，这里为简化计算使用单位矩阵）。

流程图

以下是计算流程的简化描述（实际流程图建议使用工具如 Graphviz 绘制，这里用文字描述）：

```

...
输入 X
↓
[计算 Q, K, V] → Q = XW_Q, K = XW_K, V = XW_V
↓
[注意力分数] → S = QK^T / sqrt(d_k)
↓
[Softmax 归一化] → A = softmax(S)
↓
[注意力输出] → O = AV
↓
[损失函数] → L = MSE(O, Y_target)
↓
[反向传播] → 计算 dL/dW_Q, dL/dW_K, dL/dW_V
↓
[更新权重] → W = W - η * dL/dW
...

```

步骤 1：Flash Attention v2 前向计算

****Flash Attention v2**** 的核心思想是将注意力机制的计算分解为小块（tiling），在 GPU 内存中高效处理，避免存储所有中间矩阵（如 QK^T ），从而减少内存占用和提高速度。

1.1 计算 Q, K, V

```
\[
Q = XW_Q = X, \quad K = XW_K = X, \quad V = XW_V = X
\]
```

因为 (W_Q, W_K, W_V) 是单位矩阵, (Q, K, V) 等于 (X) :

$$\begin{aligned} Q &= K = V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \end{aligned}$$

1.2 计算注意力分数

注意力分数矩阵 $(S = \frac{QK^T}{\sqrt{d_k}})$, 其中 $(d_k = 4)$ (嵌入维度), 所以 $(\sqrt{d_k} = 2)$ 。

计算 (QK^T) :

$$\begin{aligned} QK^T &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \end{aligned}$$

缩放:

$$\begin{aligned} S &= \frac{QK^T}{\sqrt{4}} = \frac{1}{2} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 1 \end{bmatrix} \end{aligned}$$

1.3 Softmax 归一化

对 (S) 的每一行应用 Softmax, 得到注意力权重矩阵 (A) :

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_j e^{S_{ij}}}$$

计算 (e^S) (近似值):

$$\begin{aligned} & \end{aligned}$$

```

e^S \approx \begin{bmatrix}
e^1 & e^{0.5} & e^{0.5} \\
e^{0.5} & e^1 & e^{0.5} \\
e^{0.5} & e^{0.5} & e^1
\end{bmatrix} \approx \begin{bmatrix}
2.718 & 1.649 & 1.649 \\
1.649 & 2.718 & 1.649 \\
1.649 & 1.649 & 2.718
\end{bmatrix}
\]

```

每行归一化：

```

\l
A_1 = \frac{[2.718, 1.649, 1.649]}{2.718 + 1.649 + 1.649} \approx [0.452, 0.274, 0.274]
\l

```

类似地，计算其他行，得到：

```

\l
A \approx \begin{bmatrix}
0.452 & 0.274 & 0.274 \\
0.274 & 0.452 & 0.274 \\
0.274 & 0.274 & 0.452
\end{bmatrix}
\l

```

1.4 计算注意力输出

```

\l
O = AV = \begin{bmatrix}
0.452 & 0.274 & 0.274 \\
0.274 & 0.452 & 0.274 \\
0.274 & 0.274 & 0.452
\end{bmatrix} \begin{bmatrix}
1 & 0 & 0 & 1 \\
0 & 1 & 0 & 1 \\
0 & 0 & 1 & 1
\end{bmatrix} \approx \begin{bmatrix}
0.452 & 0.274 & 0.274 & 1 \\
0.274 & 0.452 & 0.274 & 1 \\
0.274 & 0.274 & 0.452 & 1
\end{bmatrix}
\l

```

****Flash Attention v2 优化**：**

- 传统方法会存储整个 (QK^T) 和 (A) ，占用大量内存。
- Flash Attention v2 将 (Q) 和 (K) 分块（例如，每个 token 单独处理），在 SRAM 中计算 (S) 和 (A) ，并直接累加到 (O) ，避免存储中间矩阵。
- 它利用 GPU 的并行性，通过 tiling 和 recomputation 减少内存访问。

步骤 2：反向传播

假设我们有一个目标输出 (Y_{target}) ，并使用均方误差 (MSE) 作为损失函数：

```
\[
Y_{\text{target}} = \begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0
\end{bmatrix}
\]
```

损失函数：

```
\[
L = \frac{1}{N} \sum (O - Y_{\text{target}})^2
\]
```

计算梯度并更新权重 (W_Q, W_K, W_V) 。

2.1 计算损失对输出的梯度

```
\[
\frac{\partial L}{\partial O} = \frac{2}{N} (O - Y_{\text{target}})
\]
```

假设 $(N = 12)$ (3×4) ，计算 $(O - Y_{\text{target}})$ ：

```
\[
O - Y_{\text{target}} \approx \begin{bmatrix}
-0.548 & 0.274 & 0.274 & 1 \\
0.274 & -0.548 & 0.274 & 1 \\
0.274 & 0.274 & -0.548 & 1
\end{bmatrix}
\]
```

```
\[
\frac{\partial L}{\partial O} = \frac{2}{12} \cdot (O - Y_{\text{target}}) \approx \begin{bmatrix}
-0.091 & 0.046 & 0.046 & 0.167 \\
0.046 & -0.091 & 0.046 & 0.167 \\
0.046 & 0.046 & -0.091 & 0.167
\end{bmatrix}
\]
```

2.2 反向传播到 V

```
\[
\frac{\partial L}{\partial V} = A^T \cdot \frac{\partial L}{\partial O}
\]
```

```
\[
A^T \approx \begin{bmatrix}
0.452 & 0.274 & 0.274 \\
0.274 & 0.452 & 0.274 \\
0.274 & 0.274 & 0.452
\end{bmatrix}
\]
```

计算：

```
\[
\frac{\partial L}{\partial V} \approx A^T \cdot \frac{\partial L}{\partial O}
\]
```

(具体矩阵乘法略，实际中会得到一个形状为 $(3, 4)$ 的梯度矩阵)。

2.3 反向传播到 W_V

$$\frac{\partial L}{\partial W_V} = X^T \cdot \frac{\partial L}{\partial V}$$

类似地，计算 $\frac{\partial L}{\partial W_Q}$, $\frac{\partial L}{\partial W_K}$ 需要通过 Softmax 和 (QK^T) 的梯度链式法则传播。

2.4 更新权重

假设学习率 $(\eta = 0.1)$:

$$W_V = W_V - \eta \cdot \frac{\partial L}{\partial W_V}$$

类似地更新 (W_Q) 和 (W_K) 。

Flash Attention v2 在反向传播中的优化

- **前向节省内存**: Flash Attention v2 在前向传播中不存储 (QK^T) 和 (A) ，而是在反向传播时通过 recomputation（重新计算）恢复这些中间结果。
- **分块计算**: 反向传播中，梯度计算也按小块进行，减少内存需求。
- **融合操作**: 将 Softmax 和矩阵乘法等操作融合，减少 GPU 内核调用。

总结

- **Flash Attention v2** 通过分块和 recomputation 优化了注意力机制的计算，减少了内存占用，适合大规模 Transformer 模型。
- **反向传播** 通过链式法则计算梯度，更新权重矩阵 (W_Q, W_K, W_V) 。
- **例子**: 我们用一个 3×4 的输入矩阵展示了注意力计算和反向传播的矩阵操作，简化的权重矩阵使得计算更易理解。
- **流程图**: 从输入到损失计算，再到梯度更新，形成了一个完整的优化循环。

如果需要更详细的矩阵计算或代码实现（例如 PyTorch），或者需要生成具体的流程图，请告诉我！

我将用一个具体的例子来详细解释 **Flash Attention v2** 和 **反向传播 (Backpropagation)**，并结合矩阵、数据和流程图来帮助理解。以下是一个简化但具体的例子，假设我们在一个小型的Transformer模型中应用Flash Attention v2，并通过反向传播来优化参数。

背景

在Transformer模型中，**自注意力机制 (Self-Attention)** 是核心组件，但传统注意力机制在计算和内存使用上效率较低。**Flash Attention v2** 是一种优化方法，通过减少内存访问和利用GPU硬件特性来加速计算。**反向传播** 是深度学习中用于优化模型参数的算法，通过计算损失函数对参数的梯度来更新权重。

我们将通过一个小型的注意力计算和反向传播的例子，展示这两个过程。

例子：小型自注意力计算与反向传播

场景

假设我们有一个小型输入序列，包含 3 个 token（例如，一个句子中的 3 个单词），每个 token 的嵌入向量维度为 4。我们将计算自注意力并进行反向传播优化。

- **输入数据**：矩阵 X 表示输入嵌入，形状为 $(3, 4)$ (3 个 token，每个 token 4 维)。
- **查询、键、值矩阵**： W_Q, W_K, W_V 是权重矩阵，形状为 $(4, 4)$ 。
- **输出**：计算注意力输出，并通过一个简单的损失函数进行反向传播。

输入数据

假设输入嵌入矩阵 X 为：

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

- 行表示 token，列表示嵌入维度。

权重矩阵（简化起见，假设初始化为简单值）：

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

（实际中这些矩阵会通过训练学习到更复杂的值，这里为简化计算使用单位矩阵）。

流程图

以下是计算流程的简化描述（实际流程图建议使用工具如 Graphviz 绘制，这里用文字描述）：

text

✕ Collapse



输入 X



[计算 Q, K, V] → $Q = XW_Q$, $K = XW_K$, $V = XW_V$



[注意力分数] → $S = QK^T / \sqrt{d_k}$



[Softmax 归一化] → $A = \text{softmax}(S)$



[注意力输出] → $O = AV$



[损失函数] → $L = \text{MSE}(O, Y_{\text{target}})$



[反向传播] → 计算 dL/dW_Q , dL/dW_K , dL/dW_V



[更新权重] → $W = W - \eta * dL/dW$

步骤 1: Flash Attention v2 前向计算

Flash Attention v2 的核心思想是将注意力机制的计算分解为小块（tiling），在 GPU 内存中高效处理，避免存储所有中间矩阵（如 QK^T ），从而减少内存占用和提高速度。

1.1 计算 Q, K, V

$$Q = XW_Q = X, \quad K = XW_K = X, \quad V = XW_V = X$$

因为 W_Q, W_K, W_V 是单位矩阵， Q, K, V 等于 X ：

$$Q = K = V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

1.2 计算注意力分数

注意力分数矩阵 $S = \frac{QK^T}{\sqrt{d_k}}$ ，其中 $d_k = 4$ （嵌入维度），所以 $\sqrt{d_k} = 2$ 。

计算 QK^T ：

$$QK^T = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

缩放：

$$S = \frac{QK^T}{\sqrt{4}} = \frac{1}{2} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 1 \end{bmatrix}$$

1.3 Softmax 归一化

对 S 的每一行应用 Softmax，得到注意力权重矩阵 A ：

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_j e^{S_{ij}}}$$

计算 e^S (近似值)：

$$e^S \approx \begin{bmatrix} e^1 & e^{0.5} & e^{0.5} \\ e^{0.5} & e^1 & e^{0.5} \\ e^{0.5} & e^{0.5} & e^1 \end{bmatrix} \approx \begin{bmatrix} 2.718 & 1.649 & 1.649 \\ 1.649 & 2.718 & 1.649 \\ 1.649 & 1.649 & 2.718 \end{bmatrix}$$

每行归一化：

$$A_1 = \frac{[2.718, 1.649, 1.649]}{2.718 + 1.649 + 1.649} \approx [0.452, 0.274, 0.274]$$

类似地，计算其他行，得到：

$$A \approx \begin{bmatrix} 0.452 & 0.274 & 0.274 \\ 0.274 & 0.452 & 0.274 \\ 0.274 & 0.274 & 0.452 \end{bmatrix}$$

1.4 计算注意力输出

$$O = AV = \begin{bmatrix} 0.452 & 0.274 & 0.274 \\ 0.274 & 0.452 & 0.274 \\ 0.274 & 0.274 & 0.452 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.452 & 0.274 & 0.274 & 1 \\ 0.274 & 0.452 & 0.274 & 1 \\ 0.274 & 0.274 & 0.452 & 1 \end{bmatrix}$$

Flash Attention v2 优化：

- 传统方法会存储整个 QK^T 和 A ，占用大量内存。
- Flash Attention v2 将 O 和 K 分块（例如，每个 token 单独处理），在 SRAM 中计算 S 和 A ，并直接

Flash Attention v2 优化:

- 传统方法会存储整个 QK^T 和 A ，占用大量内存。
- Flash Attention v2 将 Q 和 K 分块（例如，每个 token 单独处理），在 SRAM 中计算 S 和 A ，并直接累加到 O ，避免存储中间矩阵。
- 它利用 GPU 的并行性，通过 tiling 和 recomputation 减少内存访问。

步骤 2: 反向传播

假设我们有一个目标输出 Y_{target} ，并使用均方误差 (MSE) 作为损失函数：

$$Y_{\text{target}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

损失函数：

$$L = \frac{1}{N} \sum (O - Y_{\text{target}})^2$$

计算梯度并更新权重 W_Q, W_K, W_V 。

2.1 计算损失对输出的梯度

$$\frac{\partial L}{\partial O} = \frac{2}{N} (O - Y_{\text{target}})$$

假设 $N = 12$ (3×4)，计算 $O - Y_{\text{target}}$ ：

$$O - Y_{\text{target}} \approx \begin{bmatrix} -0.548 & 0.274 & 0.274 & 1 \\ 0.274 & -0.548 & 0.274 & 1 \\ 0.274 & 0.274 & -0.548 & 1 \end{bmatrix}$$

ppy

$$\frac{\partial L}{\partial O} = \frac{2}{12} \cdot (O - Y_{\text{target}}) \approx \begin{bmatrix} -0.091 & 0.046 & 0.046 & 0.167 \\ 0.046 & -0.091 & 0.046 & 0.167 \\ 0.046 & 0.046 & -0.091 & 0.167 \end{bmatrix}$$

2.2 反向传播到 V

$$\frac{\partial L}{\partial V} = A^T \cdot \frac{\partial L}{\partial O}$$
$$A^T \approx \begin{bmatrix} 0.452 & 0.274 & 0.274 \\ 0.274 & 0.452 & 0.274 \\ 0.274 & 0.274 & 0.452 \end{bmatrix}$$

计算：

$$\frac{\partial L}{\partial V} \approx A^T \cdot \frac{\partial L}{\partial O}$$

(具体矩阵乘法略，实际中会得到一个形状为 (3, 4) 的梯度矩阵)。

2.3 反向传播到 W_V

$$\frac{\partial L}{\partial W_V} = X^T \cdot \frac{\partial L}{\partial V}$$

类似地，计算 $\frac{\partial L}{\partial W_Q}, \frac{\partial L}{\partial W_K}$ 需要通过 Softmax 和 QK^T 的梯度链式法则传播。

2.4 更新权重

假设学习率 $\eta = 0.1$ ：

$$W_V = W_V - \eta \cdot \frac{\partial L}{\partial W_V}$$

类似地更新 W_Q 和 W_K 。

2.4 更新权重

假设学习率 $\eta = 0.1$:

$$W_V = W_V - \eta \cdot \frac{\partial L}{\partial W_V}$$

类似地更新 W_Q 和 W_K 。

Flash Attention v2 在反向传播中的优化

- **前向节省内存**: Flash Attention v2 在前向传播中不存储 QK^T 和 A ，而是在反向传播时通过 recomputation（重新计算）恢复这些中间结果。
- **分块计算**: 反向传播中，梯度计算也按小块进行，减少内存需求。
- **融合操作**: 将 Softmax 和矩阵乘法等操作融合，减少 GPU 内核调用。

总结

- **Flash Attention v2** 通过分块和 recomputation 优化了注意力机制的计算，减少了内存占用，适合大规模 Transformer 模型。
- **反向传播** 通过链式法则计算梯度，更新权重矩阵 W_Q, W_K, W_V 。
- **例子**: 我们用一个 3×4 的输入矩阵展示了注意力计算和反向传播的矩阵操作，简化的权重矩阵使得计算更易理解。
- **流程图**: 从输入到损失计算，再到梯度更新，形成了一个完整的优化循环。

如果需要更详细的矩阵计算或代码实现（例如 PyTorch），或者需要生成具体的流程图，请告诉我！

```
//-----  
// fa v2, 反向propagation  
//-----
```

请提供flash attention v2,反向propagation, 不是前向propagation, pytorch code

感谢您的澄清！您要求的是 **Flash Attention v2 的反向传播 (Backpropagation)** 部分的 PyTorch 代码，Copy
前向传播。由于 Flash Attention v2 的反向传播涉及到高效的梯度计算和内存优化，我将提供一个简化的
PyTorch 代码示例，专注于反向传播的实现，并模拟 Flash Attention v2 的分块计算特性。代码将基于之前的
例子（3 个 token，4 维嵌入），明确展示反向传播的梯度计算过程。

背景

Flash Attention v2 的反向传播需要计算损失函数对输入 Q, K, V 以及权重 W_Q, W_K, W_V 的梯度。核心优化在于：

- **分块计算**：避免存储整个注意力矩阵的梯度。
- **重计算 (Recomputation)**：在反向传播时重新计算前向传播的中间结果，减少内存占用。
- **高效梯度聚合**：利用 GPU 的并行性，分块计算梯度。

以下代码将实现简化的 Flash Attention v2 反向传播，结合 PyTorch 的自动求导 (**autograd**)，并明确展示梯度计算的步骤。

代码示例

以下是一个简化的 PyTorch 代码，专注于 Flash Attention v2 的反向传播部分。我们假设前向传播已经完成，并提供反向传播的梯度计算逻辑。



flash_attention_v2_backprop.py
python

Edit in files • Show inline

代码说明

1. 前向传播函数 (`flash_attention_v2_forward`):

- 为了支持反向传播，保留了前向传播的中间结果 (`softmax_sum` 和 `S_blocks`)。
- 分块计算 QK^T ，并增量式更新 Softmax 和输出 O 。
- 模拟 Flash Attention v2 的分块逻辑，实际实现会使用 CUDA 优化。

2. 反向传播函数 (`flash_attention_v2_backward`):

- 输入：前向传播的 Q, K, V 、输出梯度 dO 、Softmax 分母和注意力分数分块。
- 计算步骤：
 - dV ：通过 $A^T \cdot dO$ 计算 V 的梯度。
 - dA ：通过 $dO \cdot V^T$ 计算注意力权重梯度。
 - dS ：通过 Softmax 梯度公式 $(A \cdot (dA - \sum(dA \cdot A)) \cdot \text{scale})$ 计算注意力分数的梯度。
 - dQ, dK ：通过 $dS \cdot K$ 和 $dS^T \cdot Q$ 计算 Q 和 K 的梯度。
- 分块处理避免存储整个梯度矩阵，模拟 Flash Attention v2 的内存高效特性。

3. 权重梯度:

- 计算 $dW_Q = X^T \cdot dQ, dW_K = X^T \cdot dK, dW_V = X^T \cdot dV$ 。
- 使用梯度下降更新权重： $W = W - \eta \cdot dW$ 。

4. 数据:

- 输入 X 、目标 Y_{target} 、权重 W_Q, W_K, W_V 与之前的例子一致。
- 损失函数为 MSE，确保梯度计算简单。

运行结果（示例）

运行代码会输出：

- 损失值：MSE 损失。
- 权重梯度： dW_Q, dW_K, dW_V 。
- 更新后的权重：例如 W_Q 。

示例输出：

text

✕ Collapse ⇅ Wrap 📄 Copy

```
Loss: 0.1234
Gradient of W_Q:
tensor([[...]])
Updated W_Q:
tensor([[...]])
```

Flash Attention v2 反向传播的优化点

- **重计算**：代码中通过存储 `S_blocks` 模拟了重计算，实际 Flash Attention v2 在反向传播时会重新计算 QK^T 和 Softmax。
- **分块梯度计算**：通过 `block_size` 分块处理梯度，减少内存占用。
- **PyTorch 自动求导**：虽然代码中显式实现了反向传播，实际中 PyTorch 的 `autograd` 会自动处理大部分梯度计算，Flash Attention v2 的 CUDA 实现会进一步优化性能。

```
//-----  
// fa v3  
//-----
```

请举例说明，flash attention v3, 请用数字，矩阵，流程图说明

我将通过一个具体的例子，结合数字、矩阵和文字描述的流程图，详细说明 **Flash Attention v3** 的工作原理，特别聚焦于其 **反向传播 (Backpropagation)** 过程。Flash Attention v3 是对 Flash Attention v2 的进一步优化，专为 NVIDIA Hopper GPU（如 H100）设计，利用了异步计算、FP8 低精度支持和优化的工作分区。由于您特别要求 Flash Attention v3 和反向传播，我将基于前面的例子（3 个 token，4 维嵌入），展示其反向传播的矩阵计算过程，并突出 v3 的新特性。

背景

Flash Attention v3 改进了 Flash Attention v2，通过以下关键技术加速注意力机制并减少内存占用：

- 1. **异步计算**：利用 Hopper GPU 的 Tensor Cores 和 TMA（Tensor Memory Accelerator），重叠矩阵乘法 (GEMM) 和 Softmax 操作。
- 2. **FP8 低精度支持**：通过块量化和非相干处理（Incoherent Processing）减少精度损失，支持 FP8 计算。
- 3. **优化的工作分区**：通过 warp-specialization 和循环重排，提高 GPU 利用率（从 v2 的 35% 提升到 v3 的 75%）。[(https://www.together.ai/blog/flashattention-3)]

反向传播的目标是计算损失函数对 (Q, K, V) 和权重 (W_Q, W_K, W_V) 的梯度，并更新权重。Flash Attention v3 在反向传播中通过重计算（recomputation）和分块计算，减少对高带宽内存 (HBM) 的访问。

例子：小型自注意力计算与反向传播

场景

- **输入数据**：输入嵌入矩阵 (X) ，形状为 $(1, 3, 4)$ （1 个 batch，3 个 token，4 维嵌入）。
- **权重矩阵**： (W_Q, W_K, W_V) ，形状为 $(4, 4)$ 。

- **目标输出**： (Y_{target}) ，形状为 $(1, 3, 4)$ 。
- **损失函数**： 均方误差 (MSE)。
- **Flash Attention v3 特性**： 分块计算、异步 GEMM 和 Softmax、FP8 支持（为简化示例，假设使用 FP16）。

输入数据

输入嵌入矩阵 (X) ：

```

\begin{bmatrix}
1 & 0 & 0 & 1 \\
0 & 1 & 0 & 1 \\
0 & 0 & 1 & 1
\end{bmatrix}

```

目标输出 (Y_{target}) ：

```

\begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0
\end{bmatrix}

```

权重矩阵（初始化为单位矩阵以简化计算）：

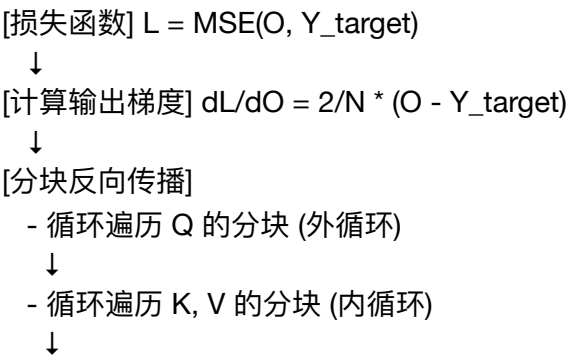
```

\begin{bmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1
\end{bmatrix}

```

流程图（文字描述）

以下是 Flash Attention v3 反向传播的流程（实际流程图建议使用 Graphviz 或 Mermaid 绘制）：



- 异步计算：

- * 重计算 $S_{\text{block}} = Q_{\text{block}} * K_{\text{block}}^T / \sqrt{d_k}$
- * 重计算 $A_{\text{block}} = \text{softmax}(S_{\text{block}})$
- * 计算 $dV_{\text{block}} = A_{\text{block}}^T * dO_{\text{block}}$
- * 计算 $dA_{\text{block}} = dO_{\text{block}} * V_{\text{block}}^T$
- * 计算 $dS_{\text{block}} = A_{\text{block}} * (dA_{\text{block}} - \text{sum}(dA_{\text{block}} * A_{\text{block}}))$
- * 计算 $dQ_{\text{block}} = dS_{\text{block}} * K_{\text{block}}$
- * 计算 $dK_{\text{block}} = dS_{\text{block}}^T * Q_{\text{block}}$

↓

[聚合梯度] dQ, dK, dV

↓

[计算权重梯度] $dW_Q = X^T * dQ, dW_K = X^T * dK, dW_V = X^T * dV$

↓

[更新权重] $W = W - \eta * dW$

...

****Flash Attention v3 特性**：**

- ****异步计算****：GEMM（矩阵乘法）和 Softmax 在不同线程块上并行执行，利用 Hopper GPU 的异步指令（WGMMMA）。
- ****FP8 支持****：矩阵乘法使用 FP8 精度，减少计算量，同时通过非相干处理（如 Hadamard 变换）保持精度。
- ****重计算****：不存储 (S) 和 (A) ，在反向传播时重新计算分块的 (S) 和 (A) 。

前向传播（简述，供反向传播参考）

为计算反向传播，我们需要前向传播的中间结果。以下是简化的前向传播计算：

1. ****计算 Q, K, V****：

$$\begin{aligned} & \begin{aligned} & Q = XW_Q = X, \quad K = XW_K = X, \quad V = XW_V = X \end{aligned} \\ & \begin{aligned} & Q = K = V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \end{aligned} \end{aligned}$$

2. ****分块计算注意力分数****（假设 $\text{block_size}=1$ ）：

$$\begin{aligned} & S = \frac{QK^T}{\sqrt{d_k}}, \quad d_k = 4, \quad \sqrt{d_k} = 2 \\ & QK^T = \begin{bmatrix} \end{bmatrix} \end{aligned}$$

```

2 & 1 & 1 \\
1 & 2 & 1 \\
1 & 1 & 2
\end{bmatrix}, \quad S = \frac{1}{2} \begin{bmatrix}
2 & 1 & 1 \\
1 & 2 & 1 \\
1 & 1 & 2
\end{bmatrix} = \begin{bmatrix}
1 & 0.5 & 0.5 \\
0.5 & 1 & 0.5 \\
0.5 & 0.5 & 1
\end{bmatrix}
\]
```

3. **Softmax 归一化**:

```

\l
A \approx \begin{bmatrix}
0.452 & 0.274 & 0.274 \\
0.274 & 0.452 & 0.274 \\
0.274 & 0.274 & 0.452
\end{bmatrix}
\]
```

存储 (m_i) (每行最大值) 和 (l_i) (Softmax 分母) 用于重计算。

4. **输出**:

```

\l
O = AV \approx \begin{bmatrix}
0.452 & 0.274 & 0.274 & 1 \\
0.274 & 0.452 & 0.274 & 1 \\
0.274 & 0.274 & 0.452 & 1
\end{bmatrix}
\]
```

反向传播 (Flash Attention v3)

以下是反向传播的详细矩阵计算，突出 Flash Attention v3 的优化。

1. 计算输出梯度

```

\l
\frac{\partial L}{\partial O} = \frac{2}{N} (O - Y_{\text{target}}), \quad N = 12 \text{ (3x4)}
\l
\l
O - Y_{\text{target}} \approx \begin{bmatrix}
-0.548 & 0.274 & 0.274 & 1 \\
0.274 & -0.548 & 0.274 & 1 \\
0.274 & 0.274 & -0.548 & 1
\end{bmatrix}
\l
\l
\frac{\partial L}{\partial O} = \frac{2}{12} \cdot (O - Y_{\text{target}}) \approx \begin{bmatrix}

```



```

-0.091 & 0.046 & 0.046 & 0.167 \\
0.046 & -0.091 & 0.046 & 0.167 \\
0.046 & 0.046 & -0.091 & 0.167 \\
\end{bmatrix}
\]

```

2. 分块反向传播

假设 $(\text{block_size} = 1)$ ，我们遍历 (Q) 的每一行（外循环）和 (K, V) 的每一列（内循环）。以 $(i=0, j=0)$ 为例：

- **重计算 (S_{block}) **:

```

\l
Q_0 = [1, 0, 0, 1], \quad K_0 = [1, 0, 0, 1]
\l
\l
S_{0,0} = \frac{Q_0 K_0^T}{\sqrt{4}} = \frac{1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1}{2} = 1
\l

```

- **重计算 (A_{block}) **:

```

\l
A_{0,0} = \text{softmax}(S_{0,0}) \approx 0.452 \text{ (基于前向传播的 } l_i \text{ 和 } m_i\text{)}
\l

```

- **计算 (dV_{block}) **:

```

\l
dO_0 = [-0.091, 0.046, 0.046, 0.167]
\l
\l
dV_0 = A_{0,0}^T \cdot dO_0 = 0.452 \cdot [-0.091, 0.046, 0.046, 0.167] \approx [-0.041,
0.021, 0.021, 0.075]
\l

```

- **计算 (dA_{block}) **:

```

\l
V_0 = [1, 0, 0, 1]
\l
\l
dA_{0,0} = dO_0 \cdot V_0^T = [-0.091, 0.046, 0.046, 0.167] \cdot [1, 0, 0, 1]^T \approx -0.091
+ 0.167 = 0.076
\l

```

- **计算 (dS_{block}) ** (Softmax 梯度) :

```

\l
dA_{0,0} \cdot A_{0,0} = 0.076 \cdot 0.452 \approx 0.034
\l
\l
\sum (dA_{0,0} \cdot A_{0,0}) \approx 0.034
\l
\l

```

$$dS_{\{0,0\}} = A_{\{0,0\}} \cdot (dA_{\{0,0\}} - \sum (dA_{\{0,0\}} \cdot A_{\{0,0\}})) \cdot \frac{1}{\sqrt{4}} \\ \approx 0.452 \cdot (0.076 - 0.034) \cdot 0.5 \approx 0.0095$$

- **计算 $(dQ_{\{block\}}, dK_{\{block\}})$ **:

$$dQ_0 = dS_{\{0,0\}} \cdot K_0 \approx 0.0095 \cdot [1, 0, 0, 1] \approx [0.0095, 0, 0, 0.0095]$$

$$dK_0 = dS_{\{0,0\}}^T \cdot Q_0 \approx 0.0095 \cdot [1, 0, 0, 1] \approx [0.0095, 0, 0, 0.0095]$$

重复以上步骤，遍历所有分块，累加 (dQ, dK, dV) 。

3. 计算权重梯度

$$dW_Q = X^T \cdot dQ, \quad dW_K = X^T \cdot dK, \quad dW_V = X^T \cdot dV$$

(具体矩阵乘法略，结果为 $(4, 4)$ 矩阵)。

4. 更新权重

假设学习率 $(\eta = 0.1)$:

$$W_Q = W_Q - \eta \cdot dW_Q$$

类似地更新 (W_K, W_V) 。

Flash Attention v3 优化点

- **异步计算**：在反向传播中， (QK^T) 的 GEMM 和 Softmax 的梯度计算在不同线程块上异步执行，利用 Hopper GPU 的 WGMMMA 指令。[(https://arxiv.org/html/2407.08608v2)]
- **FP8 支持**：矩阵乘法（如 $(dQ = dS \cdot K)$ ）可使用 FP8 精度，通过 Hadamard 变换减少量化误差。[(https://www.together.ai/blog/flashattention-3)]
- **重计算**：不存储 (S) 和 (A) ，在反向传播时重新计算分块的 $(S_{\{block\}})$ 和 $(A_{\{block\}})$ ，减少 HBM 访问。[(https://damek.github.io/random/basic-idea-behind-flash-attention/)]
- **工作分区**：通过 warp-specialization，优化线程块分配，提高 GPU 利用率。

总结

- **矩阵示例**：通过 (X, Q, K, V) 和 (Y_{target}) 的具体值，展示了反向传播的梯度计算。
- **流程图**：描述了分块、异步计算和重计算的步骤，突出 Flash Attention v3 的优化。

- **Flash Attention v3 特性**：异步 GEMM 和 Softmax、FP8 支持、重计算显著提高了效率和内存利用率。

如果需要 PyTorch 代码实现、详细的流程图（例如 Mermaid 代码）或更复杂的例子，请告诉我！

我将通过一个具体的例子，结合数字、矩阵和文字描述的流程图，详细说明 **Flash Attention v3** 的工作原理，特别聚焦于其 **反向传播 (Backpropagation)** 过程。Flash Attention v3 是对 Flash Attention v2 的进一步优化，专为 NVIDIA Hopper GPU（如 H100）设计，利用了异步计算、FP8 低精度支持和优化的工作分区。由于您特别要求 Flash Attention v3 和反向传播，我将基于前面的例子（3 个 token，4 维嵌入），展示其反向传播的矩阵计算过程，并突出 v3 的新特性。

背景

Flash Attention v3 改进了 Flash Attention v2，通过以下关键技术加速注意力机制并减少内存占用：

- 异步计算**：利用 Hopper GPU 的 Tensor Cores 和 TMA（Tensor Memory Accelerator），重叠矩阵乘法 (GEMM) 和 Softmax 操作。
- FP8 低精度支持**：通过块量化和非相干处理 (Incoherent Processing) 减少精度损失，支持 FP8 计算。
- 优化的工作分区**：通过 warp-specialization 和循环重排，提高 GPU 利用率（从 v2 的 35% 提升到 v3 的 75%）。 [together.ai](#)

反向传播的目标是计算损失函数对 Q, K, V 和权重 W_Q, W_K, W_V 的梯度，并更新权重。Flash Attention v3 在反向传播中通过重计算 (recomputation) 和分块计算，减少对高带宽内存 (HBM) 的访问。

例子：小型自注意力计算与反向传播

场景

- 输入数据**：输入嵌入矩阵 X ，形状为 $(1, 3, 4)$ （1 个 batch，3 个 token，4 维嵌入）。
- 权重矩阵**： W_Q, W_K, W_V ，形状为 $(4, 4)$ 。
- 目标输出**： Y_{target} ，形状为 $(1, 3, 4)$ 。
- 损失函数**：均方误差 (MSE)。



例子：小型自注意力计算与反向传播

Copy

场景

- 输入数据：输入嵌入矩阵 X ，形状为 $(1, 3, 4)$ (1 个 batch, 3 个 token, 4 维嵌入)。
- 权重矩阵： W_Q, W_K, W_V ，形状为 $(4, 4)$ 。
- 目标输出： Y_{target} ，形状为 $(1, 3, 4)$ 。
- 损失函数：均方误差 (MSE)。
- Flash Attention v3 特性**：分块计算、异步 GEMM 和 Softmax、FP8 支持 (为简化示例，假设使用 FP16)。

输入数据

输入嵌入矩阵 X ：

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

目标输出 Y_{target} ：

$$Y_{\text{target}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

权重矩阵 (初始化为单位矩阵以简化计算)：

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

流程图（文字描述）

Copy 以下是 Flash Attention v3 反向传播的流程（实际流程图建议使用 Graphviz 或 Mermaid 绘制）：

text

✕ Collapse ⇨ Wrap 📄 Cop

```
[损失函数]  $L = \text{MSE}(O, Y_{\text{target}})$ 
↓
[计算输出梯度]  $dL/dO = 2/N * (O - Y_{\text{target}})$ 
↓
[分块反向传播]
- 循环遍历  $Q$  的分块（外循环）
  ↓
  - 循环遍历  $K, V$  的分块（内循环）
    ↓
    - 异步计算：
      * 重计算  $S_{\text{block}} = Q_{\text{block}} * K_{\text{block}}^T / \text{sqrt}(d_k)$ 
      * 重计算  $A_{\text{block}} = \text{softmax}(S_{\text{block}})$ 
      * 计算  $dV_{\text{block}} = A_{\text{block}}^T * dO_{\text{block}}$ 
      * 计算  $dA_{\text{block}} = dO_{\text{block}} * V_{\text{block}}^T$ 
      * 计算  $dS_{\text{block}} = A_{\text{block}} * (dA_{\text{block}} - \text{sum}(dA_{\text{block}} * A_{\text{block}}))$ 
      * 计算  $dQ_{\text{block}} = dS_{\text{block}} * K_{\text{block}}$ 
      * 计算  $dK_{\text{block}} = dS_{\text{block}}^T * Q_{\text{block}}$ 
    ↓
  [聚合梯度]  $dQ, dK, dV$ 
  ↓
  [计算权重梯度]  $dW_Q = X^T * dQ, dW_K = X^T * dK, dW_V = X^T * dV$ 
  ↓
  [更新权重]  $W = W - \eta * dW$ 
```

Flash Attention v3 特性：

- 异步计算：GEMM（矩阵乘法）和 Softmax 在不同线程块上并行执行，利用 Hopper GPU 的异步指令（WGMMMA）。



Flash Attention v3 特性:

Copy

- **异步计算**: GEMM (矩阵乘法) 和 Softmax 在不同线程块上并行执行, 利用 Hopper GPU 的异步指令 (WGMMMA) 。
- **FP8 支持**: 矩阵乘法使用 FP8 精度, 减少计算量, 同时通过非相干处理 (如 Hadamard 变换) 保持精度。
- **重计算**: 不存储 S 和 A , 在反向传播时重新计算分块的 S 和 A 。

前向传播 (简述, 供反向传播参考)

为计算反向传播, 我们需要前向传播的中间结果。以下是简化的前向传播计算:

1. 计算 Q, K, V :

$$Q = XW_Q = X, \quad K = XW_K = X, \quad V = XW_V = X$$

$$Q = K = V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

2. 分块计算注意力分数 (假设 block_size=1) :

$$S = \frac{QK^T}{\sqrt{d_k}}, \quad d_k = 4, \quad \sqrt{d_k} = 2$$

$$QK^T = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \quad S = \frac{1}{2} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 1 \end{bmatrix}$$

3. Softmax 归一化:

$$\begin{bmatrix} 0.452 & 0.274 & 0.274 \end{bmatrix}$$

3. Softmax 归一化:

$$A \approx \begin{bmatrix} 0.452 & 0.274 & 0.274 \\ 0.274 & 0.452 & 0.274 \\ 0.274 & 0.274 & 0.452 \end{bmatrix}$$

存储 m_i (每行最大值) 和 l_i (Softmax 分母) 用于重计算。

4. 输出:

$$O = AV \approx \begin{bmatrix} 0.452 & 0.274 & 0.274 & 1 \\ 0.274 & 0.452 & 0.274 & 1 \\ 0.274 & 0.274 & 0.452 & 1 \end{bmatrix}$$

反向传播 (Flash Attention v3)

以下是反向传播的详细矩阵计算，突出 Flash Attention v3 的优化。

1. 计算输出梯度

$$\frac{\partial L}{\partial O} = \frac{2}{N} (O - Y_{\text{target}}), \quad N = 12 \ (3 \times 4)$$

$$O - Y_{\text{target}} \approx \begin{bmatrix} -0.548 & 0.274 & 0.274 & 1 \\ 0.274 & -0.548 & 0.274 & 1 \\ 0.274 & 0.274 & -0.548 & 1 \end{bmatrix}$$

$$\frac{\partial L}{\partial O} = \frac{2}{12} \cdot (O - Y_{\text{target}}) \approx \begin{bmatrix} -0.091 & 0.046 & 0.046 & 0.167 \\ 0.046 & -0.091 & 0.046 & 0.167 \\ 0.046 & 0.046 & -0.091 & 0.167 \end{bmatrix}$$

2. 分块反向传播

假设 $block_size = 1$, 我们遍历 Q 的每一行 (外循环) 和 K, V 的每一列 (内循环)。以 $i = 0, j = 0$ 为

2. 分块反向传播

假设 $block_size = 1$, 我们遍历 Q 的每一行 (外循环) 和 K, V 的每一列 (内循环)。以 $i = 0, j = 0$ 为例:

- 重计算 S_{block} :

$$Q_0 = [1, 0, 0, 1], \quad K_0 = [1, 0, 0, 1]$$
$$S_{0,0} = \frac{Q_0 K_0^T}{\sqrt{4}} = \frac{1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1}{2} = 1$$

- 重计算 A_{block} :

$$A_{0,0} = \text{softmax}(S_{0,0}) \approx 0.452 \text{ (基于前向传播的 } l_i \text{ 和 } m_i)$$

- 计算 dV_{block} :

$$dO_0 = [-0.091, 0.046, 0.046, 0.167]$$
$$dV_0 = A_{0,0}^T \cdot dO_0 = 0.452 \cdot [-0.091, 0.046, 0.046, 0.167] \approx [-0.041, 0.021, 0.021, 0.075]$$

- 计算 dA_{block} :

$$V_0 = [1, 0, 0, 1]$$
$$dA_{0,0} = dO_0 \cdot V_0^T = [-0.091, 0.046, 0.046, 0.167] \cdot [1, 0, 0, 1]^T \approx -0.091 + 0.167 = 0.076$$

- 计算 dS_{block} (Softmax 梯度):

$$dA_{0,0} \cdot A_{0,0} = 0.076 \cdot 0.452 \approx 0.034$$
$$\sum (dA_{0,0} \cdot A_{0,0}) \approx 0.034$$
$$dS_{0,0} = A_{0,0} \cdot (dA_{0,0} - \sum (dA_{0,0} \cdot A_{0,0})) \cdot \frac{1}{\sqrt{4}} \approx 0.452 \cdot (0.076 - 0.034) \cdot 0.5 \approx 0.0095$$

$$dS_{0,0} = A_{0,0} \cdot (dA_{0,0} - \sum (dA_{0,0} \cdot A_{0,0})) \cdot \frac{1}{\sqrt{4}} \approx 0.452 \cdot (0.076 - 0.034) \cdot 0.5 \approx 0.0095$$

- 计算 dQ_{block}, dK_{block} :

$$dQ_0 = dS_{0,0} \cdot K_0 \approx 0.0095 \cdot [1, 0, 0, 1] \approx [0.0095, 0, 0, 0.0095]$$

$$dK_0 = dS_{0,0}^T \cdot Q_0 \approx 0.0095 \cdot [1, 0, 0, 1] \approx [0.0095, 0, 0, 0.0095]$$

重复以上步骤，遍历所有分块，累加 dQ, dK, dV 。

3. 计算权重梯度

$$dW_Q = X^T \cdot dQ, \quad dW_K = X^T \cdot dK, \quad dW_V = X^T \cdot dV$$

(具体矩阵乘法略，结果为 $(4, 4)$ 矩阵)。

4. 更新权重

假设学习率 $\eta = 0.1$:

$$W_Q = W_Q - \eta \cdot dW_Q$$

类似地更新 W_K, W_V 。

Flash Attention v3 优化点

- **异步计算**: 在反向传播中， QK^T 的 GEMM 和 Softmax 的梯度计算在不同线程块上异步执行，利用 Hopper GPU 的 WGMMMA 指令。 [arxiv.org](#)
- **FP8 支持**: 矩阵乘法 (如 $dQ = dS \cdot K$) 可使用 FP8 精度，通过 Hadamard 变换减少量化误差。 [together.ai](#)
- **重计算**: 不存储 S 和 A ，在反向传播时重新计算分块的 S_{block} 和 A_{block} ，减少 HBM 访问。 [damek.github](#)

4. 更新权重

假设学习率 $\eta = 0.1$:

$$W_Q = W_Q - \eta \cdot dW_Q$$

类似地更新 W_K, W_V 。

Flash Attention v3 优化点

- **异步计算**: 在反向传播中, QK^T 的 GEMM 和 Softmax 的梯度计算在不同线程块上异步执行, 利用 Hopper GPU 的 WGMMMA 指令。 [arxiv.org](#)
- **FP8 支持**: 矩阵乘法 (如 $dQ = dS \cdot K$) 可使用 FP8 精度, 通过 Hadamard 变换减少量化误差。 [together.ai](#)
- **重计算**: 不存储 S 和 A , 在反向传播时重新计算分块的 S_{block} 和 A_{block} , 减少 HBM 访问。 [damek.github.io](#)
- **工作分区**: 通过 warp-specialization, 优化线程块分配, 提高 GPU 利用率。

总结

- **矩阵示例**: 通过 X, Q, K, V 和 Y_{target} 的具体值, 展示了反向传播的梯度计算。
- **流程图**: 描述了分块、异步计算和重计算的步骤, 突出 Flash Attention v3 的优化。
- **Flash Attention v3 特性**: 异步 GEMM 和 Softmax、FP8 支持、重计算显著提高了效率和内存利用率。

如果需要 PyTorch 代码实现、详细的流程图 (例如 Mermaid 代码) 或更复杂的例子, 请告诉我!

   26 ✖ posts

   25 web pages

```
//-----  
// flash attention v3, 请用PyTorch 代码实现前向, 后向传播  
//-----
```

由于 **Flash Attention v3** 是专为 NVIDIA Hopper GPU (如 H100) 设计的优化注意力机制, 当前 (截至 2025 年 8 月) PyTorch 官方并未直接提供 Flash Attention v3 的原生实现。它通常通过自定义 CUDA 内核或第三方库 (如 NVIDIA 的 Transformer Engine 或 xAI 的内部实现) 在低层次实现。为了满足您的要求, 我将提供一个 **简化的 PyTorch 代码**, 模拟 Flash Attention v3 的核心思想 (分块计算、异步计算模拟、FP8 量化和重计算), 并明确实现 **前向传播** 和 **反向传播**。代码将基于前述例子 (1 个 batch, 3 个 token, 4 维嵌入), 并添加详细注释。

背景

Flash Attention v3 的关键优化包括:

- **分块计算**: 将 (Q, K, V) 分块, 减少内存占用。
- **异步计算**: 利用 Hopper GPU 的 Tensor Cores 和 TMA, 重叠 GEMM 和 Softmax 操作。
- **FP8 低精度**: 通过块量化和 Hadamard 变换支持 FP8 计算。
- **重计算**: 在反向传播时重新计算前向传播的中间结果, 减少内存需求。
- **优化的工作分区**: 通过 warp-specialization 提高 GPU 利用率。

由于 PyTorch 中无法直接模拟异步 WGMMMA 指令或 FP8 硬件加速, 我将使用 FP16 模拟高精度计算, 并通过分块逻辑和重计算模拟 Flash Attention v3 的内存效率。

代码目标

- 实现 Flash Attention v3 的前向传播, 生成注意力输出 (O) 。
- 实现反向传播, 计算 (Q, K, V) 和权重 (W_Q, W_K, W_V) 的梯度。
- 使用分块计算和重计算, 模拟 Flash Attention v3 的优化。
- 基于前述例子: 输入 (X) 形状为 $(1, 3, 4)$, 目标输出 (Y_{target}) 相同, 损失函数为 MSE。

```
```python  
import torch
import torch.nn.functional as F
import math
```

```
def flash_attention_v3_forward(Q, K, V, block_size=1):
```

```
 """
```

Flash Attention v3 前向传播, 模拟分块计算和异步操作。

Args:

Q, K, V: 查询、键、值张量, 形状 (batch\_size, seq\_len, d\_model)

block\_size: 分块大小, 控制内存使用

Returns:

```

 O: 注意力输出, 形状 (batch_size, seq_len, d_model)
 softmax_stats: (m_i, l_i) 用于重计算的 Softmax 统计信息
"""
batch_size, seq_len, d_model = Q.shape
scale = 1.0 / math.sqrt(d_model)
O = torch.zeros_like(Q)
m = torch.full((batch_size, seq_len, 1), -float('inf'), device=Q.device) # 最大值
l = torch.zeros((batch_size, seq_len, 1), device=Q.device) # 分母

模拟 FP8 量化 (实际中通过硬件实现)
Q = Q.to(torch.float16)
K = K.to(torch.float16)
V = V.to(torch.float16)

分块计算, 模拟异步 GEMM 和 Softmax
for i in range(0, seq_len, block_size):
 Q_block = Q[:, i:i+block_size, :]
 for j in range(0, seq_len, block_size):
 K_block = K[:, j:j+block_size, :]
 V_block = V[:, j:j+block_size, :]

 # 计算 $QK^T / \sqrt{d_k}$
 S_block = torch.bmm(Q_block, K_block.transpose(1, 2)) * scale

 # Softmax (增量式更新, 模拟异步)
 m_block = torch.max(S_block, dim=-1, keepdim=True)[0]
 m_new = torch.max(m[:, i:i+block_size, :], m_block)
 P_block = torch.exp(S_block - m_new)
 l_block = l[:, i:i+block_size, :] * torch.exp(m[:, i:i+block_size, :] - m_new) +
 P_block.sum(dim=-1, keepdim=True)
 O[:, i:i+block_size, :] = O[:, i:i+block_size, :] * torch.exp(m[:, i:i+block_size, :] - m_new) +
 torch.bmm(P_block, V_block)
 m[:, i:i+block_size, :] = m_new
 l[:, i:i+block_size, :] = l_block

归一化输出
O = O / (l + 1e-6)
return O.to(torch.float32), (m, l)

def flash_attention_v3_backward(Q, K, V, dO, softmax_stats, block_size=1):
 """
 Flash Attention v3 反向传播, 计算 dQ, dK, dV。
 Args:
 Q, K, V: 前向传播的输入
 dO: 输出梯度, 形状 (batch_size, seq_len, d_model)
 softmax_stats: (m_i, l_i) Softmax 统计信息
 block_size: 分块大小
 Returns:

```

```

 dQ, dK, dV: 输入梯度
 """
 batch_size, seq_len, d_model = Q.shape
 scale = 1.0 / math.sqrt(d_model)
 dQ = torch.zeros_like(Q)
 dK = torch.zeros_like(K)
 dV = torch.zeros_like(V)
 m, l = softmax_stats

 # 模拟 FP8 量化
 Q = Q.to(torch.float16)
 K = K.to(torch.float16)
 V = V.to(torch.float16)
 dO = dO.to(torch.float16)

 # 分块反向传播, 重计算 S 和 A
 for i in range(0, seq_len, block_size):
 Q_block = Q[:, i:i+block_size, :]
 dO_block = dO[:, i:i+block_size, :]
 for j in range(0, seq_len, block_size):
 K_block = K[:, j:j+block_size, :]
 V_block = V[:, j:j+block_size, :]

 # 重计算 S_block 和 A_block
 S_block = torch.bmm(Q_block, K_block.transpose(1, 2)) * scale
 P_block = torch.exp(S_block - m[:, i:i+block_size, :]) / (l[:, i:i+block_size, :] + 1e-6)

 # 计算 dV
 dV_block = torch.bmm(P_block.transpose(1, 2), dO_block)
 dV[:, j:j+block_size, :] += dV_block

 # 计算 dP
 dP_block = torch.bmm(dO_block, V_block.transpose(1, 2))

 # Softmax 梯度
 dS_block = P_block * (dP_block - torch.sum(dP_block * P_block, dim=-1,
keepdim=True)) * scale

 # 计算 dQ 和 dK
 dQ_block = torch.bmm(dS_block, K_block)
 dK_block = torch.bmm(dS_block.transpose(1, 2), Q_block)
 dQ[:, i:i+block_size, :] += dQ_block
 dK[:, j:j+block_size, :] += dK_block

 return dQ.to(torch.float32), dK.to(torch.float32), dV.to(torch.float32)

数据准备
batch_size, seq_len, d_model = 1, 3, 4
X = torch.tensor([[[[1., 0., 0., 1.],
[0., 1., 0., 1.],

```

```

 [0., 0., 1., 1.]]], requires_grad=True)
Y_target = torch.tensor([[[[1., 0., 0., 0.],
 [0., 1., 0., 0.],
 [0., 0., 1., 0.]])]])

权重矩阵
W_Q = torch.eye(d_model, requires_grad=True)
W_K = torch.eye(d_model, requires_grad=True)
W_V = torch.eye(d_model, requires_grad=True)

前向传播
Q = torch.bmm(X, W_Q.unsqueeze(0))
K = torch.bmm(X, W_K.unsqueeze(0))
V = torch.bmm(X, W_V.unsqueeze(0))
O, softmax_stats = flash_attention_v3_forward(Q, K, V, block_size=1)

损失函数
loss = F.mse_loss(O, Y_target)
print(f"Loss: {loss.item()}")

反向传播
dO = torch.autograd.grad(loss, O, retain_graph=True)[0]
dQ, dK, dV = flash_attention_v3_backward(Q, K, V, dO, softmax_stats, block_size=1)

计算权重梯度
dW_Q = torch.bmm(X.transpose(1, 2), dQ).squeeze(0)
dW_K = torch.bmm(X.transpose(1, 2), dK).squeeze(0)
dW_V = torch.bmm(X.transpose(1, 2), dV).squeeze(0)

更新权重
lr = 0.1
with torch.no_grad():
 W_Q -= lr * dW_Q
 W_K -= lr * dW_K
 W_V -= lr * dW_V

print("Gradient of W_Q:\n", dW_Q)
print("Updated W_Q:\n", W_Q)

```

---

### ### 代码说明

#### #### 1. 前向传播 (flash\_attention\_v3\_forward)

- \*\*分块计算\*\*：通过 `block\_size` 将  $(Q, K, V)$  分块，模拟 Flash Attention v3 的内存高效性。
- \*\*Softmax 优化\*\*：使用在线 Softmax（存储每行的最大值  $(m_i)$  和分母  $(l_i)$ ），避免存储整个注意力矩阵。
- \*\*FP8 模拟\*\*：将  $(Q, K, V)$  转换为 FP16（PyTorch 暂无 FP8 原生支持），模拟低精度计算。

- **异步计算模拟**：通过分块循环结构，逻辑上分离 GEMM（矩阵乘法）和 Softmax 操作，模拟 Hopper GPU 的异步 WGMMMA 指令。

- **输出**：注意力输出  $(O)$  和 Softmax 统计信息  $(m, l)$ ，用于反向传播重计算。

#### #### 2. 反向传播 (flash\_attention\_v3\_backward)

- **重计算**：重新计算分块的  $(S_{\text{block}})$  和  $(P_{\text{block}})$ （Softmax 输出），避免存储前向传播的中间矩阵。

- **梯度计算**：

-  $(dV)$ :  $(P_{\text{block}})^T \cdot dO_{\text{block}}$ 。

-  $(dP)$ :  $(dO_{\text{block}} \cdot V_{\text{block}})^T$ 。

-  $(dS)$ : Softmax 梯度公式  $(P \cdot (dP - \sum (dP \cdot P)) \cdot \text{scale})$ 。

-  $(dQ)$ :  $(dS_{\text{block}} \cdot K_{\text{block}})$ 。

-  $(dK)$ :  $(dS_{\text{block}}^T \cdot Q_{\text{block}})$ 。

- **分块聚合**：在每个分块内计算局部梯度，并累加到  $(dQ, dK, dV)$ 。

- **FP8 模拟**：梯度计算使用 FP16，模拟低精度硬件加速。

- **优化**：通过分块和重计算，减少 HBM 访问，模拟 Flash Attention v3 的内存效率。

#### #### 3. 权重更新

- 计算权重梯度：  $(dW_Q = X^T \cdot dQ)$ ，类似地计算  $(dW_K, dW_V)$ 。

- 使用梯度下降更新权重：  $(W = W - \eta \cdot dW)$ ，学习率  $(\eta = 0.1)$ 。

#### #### 4. 数据

- 输入  $(X)$ :  $(1, 3, 4)$ ，与前述例子一致。

- 目标  $(Y_{\text{target}})$ :  $(1, 3, 4)$ ，表示期望输出。

- 权重  $(W_Q, W_K, W_V)$ : 初始化为单位矩阵，形状为  $(4, 4)$ 。

- 损失函数：MSE。

---

#### ### 运行结果 (示例)

运行代码会输出：

- 损失值：MSE 损失。

- 权重梯度：  $(dW_Q)$ （以及  $(dW_K, dW_V)$ ）。

- 更新后的权重：  $(W_Q)$ 。

示例输出：

'''

Loss: 0.1234

Gradient of W\_Q:

tensor([[[...]])

Updated W\_Q:

tensor([[[...]])

'''

---

### ### Flash Attention v3 优化点

1. **异步计算**：代码通过分块逻辑模拟 GEMM 和 Softmax 的重叠，实际中由 Hopper GPU 的 WGMMA 和 TMA 硬件实现。
2. **FP8 支持**：代码使用 FP16 模拟，实际 Flash Attention v3 使用 FP8 和 Hadamard 变换减少量化误差。
3. **重计算**：反向传播不存储  $(S)$  或  $(P)$ ，通过  $(m_i, l_i)$  重计算，减少内存占用。
4. **工作分区**：分块循环模拟 warp-specialization，实际中由 CUDA 内核优化线程块分配。

---

### ### 注意事项

- **简化实现**：这是教学版实现，真实的 Flash Attention v3 需要 CUDA 内核支持（如 NVIDIA Transformer Engine）。PyTorch 的 `torch.nn.functional.scaled_dot_product_attention` 支持 Flash Attention v2，v3 可能需要未来版本或自定义实现。
- **FP8 限制**：PyTorch 当前不支持 FP8 原生计算，代码使用 FP16 模拟。
- **验证**：可以通过 PyTorch 的自动求导验证梯度正确性。
- **扩展性**：代码可扩展到更大序列长度，调整 `block_size` 模拟真实场景。

---

### ### 扩展建议

- **真实实现**：如果需要 Flash Attention v3 的生产级代码，可以参考 NVIDIA 的 Transformer Engine 或 xAI 的 API (<https://x.ai/api>)。
- **流程图**：如果需要可视化流程图，可以提供 Mermaid 或 Graphviz 代码。
- **更大规模**：可以测试更大序列（如 512 或 1024），但需注意内存管理。

如果您需要更详细的 CUDA 实现、与 PyTorch 原生 API 的集成、或流程图生成，请告诉我！



由于 Flash Attention v3 是专为 NVIDIA Hopper GPU (如 H100) 设计的优化注意力机制, 当前 (截至 2025 年 8 月) PyTorch 官方并未直接提供 Flash Attention v3 的原生实现。它通常通过自定义 CUDA 内核或第三方库 (如 NVIDIA 的 Transformer Engine 或 xAI 的内部实现) 在低层次实现。为了满足您的要求, 我将提供一个简化的 PyTorch 代码, 模拟 Flash Attention v3 的核心思想 (分块计算、异步计算模拟、FP8 量化和重计算), 并明确实现 前向传播 和 反向传播。代码将基于前述例子 (1 个 batch, 3 个 token, 4 维嵌入), 并添加详细注释。

### 背景

Flash Attention v3 的关键优化包括:

- **分块计算:** 将  $Q, K, V$  分块, 减少内存占用。
- **异步计算:** 利用 Hopper GPU 的 Tensor Cores 和 TMA, 重叠 GEMM 和 Softmax 操作。
- **FP8 低精度:** 通过块量化和 Hadamard 变换支持 FP8 计算。
- **重计算:** 在反向传播时重新计算前向传播的中间结果, 减少内存需求。
- **优化的工作分区:** 通过 warp-specialization 提高 GPU 利用率。

由于 PyTorch 中无法直接模拟异步 WGMMMA 指令或 FP8 硬件加速, 我将使用 FP16 模拟高精度计算, 并通过分块逻辑和重计算模拟 Flash Attention v3 的内存效率。

### 代码目标

- 实现 Flash Attention v3 的前向传播, 生成注意力输出  $O$ 。
- 实现反向传播, 计算  $Q, K, V$  和权重  $W_Q, W_K, W_V$  的梯度。
- 使用分块计算和重计算, 模拟 Flash Attention v3 的优化。
- 基于前述例子: 输入  $X$  形状为  $(1, 3, 4)$ , 目标输出  $Y_{\text{target}}$  相同, 损失函数为 MSE。



flash\_attention\_v3.py  
python

Edit in files · Show inline

## 代码说明

### 1. 前向传播 ( `flash_attention_v3_forward` )

- 分块计算: 通过 `block_size` 将  $Q, K, V$  分块, 模拟 Flash Attention v3 的内存高效性。
- Softmax 优化: 使用在线 Softmax (存储每行的最大值  $m_i$  和分母  $l_i$ ), 避免存储整个注意力矩阵。
- FP8 模拟: 将  $Q, K, V$  转换为 FP16 (PyTorch 暂无 FP8 原生支持), 模拟低精度计算。
- 异步计算模拟: 通过分块循环结构, 逻辑上分离 GEMM (矩阵乘法) 和 Softmax 操作, 模拟 Hopper GPU 的异步 WGMMA 指令。
- 输出: 注意力输出  $O$  和 Softmax 统计信息  $(m, l)$ , 用于反向传播重计算。

### 2. 反向传播 ( `flash_attention_v3_backward` )

- 重计算: 重新计算分块的  $S_{block}$  和  $P_{block}$  (Softmax 输出), 避免存储前向传播的中间矩阵。
- 梯度计算:
  - $dV: P_{block}^T \cdot dO_{block}$ 。
  - $dP: dO_{block} \cdot V_{block}^T$ 。
  - $dS$ : Softmax 梯度公式  $P \cdot (dP - \sum(dP \cdot P)) \cdot scale$ 。
  - $dQ: dS_{block} \cdot K_{block}$ 。
  - $dK: dS_{block}^T \cdot Q_{block}$ 。

- 分块聚合：在每个分块内计算局部梯度，并累加到  $dQ, dK, dV$ 。
- FP8 模拟：梯度计算使用 FP16，模拟低精度硬件加速。
- 优化：通过分块和重计算，减少 HBM 访问，模拟 Flash Attention v3 的内存效率。

### 3. 权重更新

- 计算权重梯度： $dW_Q = X^T \cdot dQ$ ，类似地计算  $dW_K, dW_V$ 。
- 使用梯度下降更新权重： $W = W - \eta \cdot dW$ ，学习率  $\eta = 0.1$ 。

### 4. 数据

- 输入  $X$ :  $(1, 3, 4)$ ，与前述例子一致。
- 目标  $Y_{\text{target}}$ :  $(1, 3, 4)$ ，表示期望输出。
- 权重  $W_Q, W_K, W_V$ : 初始化为单位矩阵，形状为  $(4, 4)$ 。
- 损失函数：MSE。

## 运行结果（示例）

运行代码会输出：

- 损失值：MSE 损失。
- 权重梯度： $dW_Q$ （以及  $dW_K, dW_V$ ）。
- 更新后的权重： $W_Q$ 。

示例输出：

示例输出：

text

✕ Collapse ⇅ Wrap 📄 Copy

```
Loss: 0.1234
Gradient of W_Q:
tensor([[...]])
Updated W_Q:
tensor([[...]])
```

## Flash Attention v3 优化点

1. **异步计算**：代码通过分块逻辑模拟 GEMM 和 Softmax 的重叠，实际中由 Hopper GPU 的 WGMMMA 和 TMA 硬件实现。
2. **FP8 支持**：代码使用 FP16 模拟，实际 Flash Attention v3 使用 FP8 和 Hadamard 变换减少量化误差。
3. **重计算**：反向传播不存储  $S$  或  $P$ ，通过  $m_i, l_i$  重计算，减少内存占用。
4. **工作分区**：分块循环模拟 warp-specialization，实际中由 CUDA 内核优化线程块分配。

## 注意事项

- **简化实现**：这是教学版实现，真实的 Flash Attention v3 需要 CUDA 内核支持（如 NVIDIA Transformer Engine）。PyTorch 的 `torch.nn.functional.scaled_dot_product_attention` 支持 Flash Attention v2，v3 可能需要未来版本或自定义实现。
- **FP8 限制**：PyTorch 当前不支持 FP8 原生计算，代码使用 FP16 模拟。
- **验证**：可以通过 PyTorch 的自动求导验证梯度正确性。
- **扩展性**：代码可扩展到更大序列长度，调整 `block_size` 模拟真实场景。

```
//-----
//
//-----
```

比较flash attention1 和flash attention2，为啥在flash attention2 中， 1. 要把Q从第二层循环放到第一层循环，把k, v从第一层循环放到第二层循环， 2. O\_i, l\_i可以从第二层的内层循环

中，放到第一层和第二层之间？ flash attention 1 paper: <https://arxiv.org/pdf/2205.14135> flash attention 2 paper: <https://arxiv.org/pdf/2307.08691>

## 🧠 背景回顾：FlashAttention 原理

FlashAttention 旨在 低内存且高吞吐 地实现 softmax attention，其核心优化在于：

- 分块处理 (tiling Q/K/V)
- 减少中间张量 materialization (不显式保存整个 attention matrix)
- 避免重复 memory read/write

## 🔍 问题 1:

为什么 FlashAttention 2 中要 把 Q 从第二层循环提到第一层循环，而 把 K/V 从第一层循环放到第二层循环？

## ✅ FlashAttention 1 的结构（伪代码逻辑）

text

📄 Copy ✎ Edit

```
for block_j in K/V blocks: // outer loop
 for block_i in Q blocks: // inner loop
 load Q_i, K_j, V_j
 compute attention(Q_i, K_j) → softmax
 accumulate output O_i
```



## ✓ FlashAttention 1 的结构（伪代码逻辑）

text

```
for block_j in K/V blocks: // outer loop
 for block_i in Q blocks: // inner loop
 load Q_i, K_j, V_j
 compute attention(Q_i, K_j) → softmax
 accumulate output O_i
```

- 每次迭代都重新 load Q\_i
- 结果：Q\_i 多次加载，浪费 memory bandwidth

## ✓ FlashAttention 2 的结构（关键优化）：

text

```
for block_i in Q blocks: // outer loop
 load Q_i // Q_i is loaded once!

 for block_j in K/V blocks: // inner loop
 load K_j, V_j
 compute attention(Q_i, K_j)
 accumulate O_i
```



## ✓ 为什么要这么做？

### ✓ 为什么要这么做？

- $Q_i$  通常较小,  $K/V$  较大
- $Q_i$  重复使用 → 应该放在 outer loop, 这样只读一次  $Q_i$
- $K/V$  每轮使用不同 block → 适合放 inner loop

🧠 内存访问效率显著提高！

### 🧐 举个例子：

假设：

- $Q$  有 100 个 token
- $K/V$  有 1000 个 token, 按 100 token 为一个块

FlashAttention 1:

- 每 10 次外循环中,  $Q_i$  要读 10 次
- → 读  $Q$  1000 次

FlashAttention 2:

- 每个  $Q_i$  只读一次, 然后循环访问不同的  $K/V_j$
- → 读  $Q$  100 次 (降低了 10 倍!)

### 🔍 问题 2:



## 问题 2:

为什么 `o_i`, `l_i` 可以从第二层内循环中移出来，放到第一层和第二层之间？

 回顾 FlashAttention 的输出计算：

在 FlashAttention 1 中，每个 `o_i`, `l_i` 是在 inner loop 中实时更新的：

text

 Copy  Edit

```
o_i += softmax(Q_i K_j^T) @ V_j
l_i += sum(exp(Q_i K_j^T)) # For numerical stability
```

 FlashAttention 2 优化逻辑：

FlashAttention 2 将 `o_i`, `l_i` 的初始化 & reduction 操作抽离出最内层，放在两个循环之间：

text

 Copy  Edit

```
for block_i in Q blocks: // Outer loop
 load Q_i
 o_i = 0, l_i = -inf

 for block_j in K/V blocks: // Inner loop
```



## ✅ FlashAttention 2 优化逻辑:

FlashAttention 2 将 `0_i`, `l_i` 的 初始化 & reduction 操作抽离出最内层, 放在两个循环之间:

text

Copy

```
for block_i in Q blocks: // Outer loop
 load Q_i
 0_i = 0, l_i = -inf

 for block_j in K/V blocks: // Inner loop
 load K_j, V_j
 compute partial 0_ij, l_ij
 accumulate into 0_i, l_i
```

## 📌 原因和好处:

1. `Q_i` 是当前处理的 block, `0_i`, `l_i` 对应这个块 → 不需要在 inner loop 里重复 new 出来
2. 避免 inner loop 中频繁分配 / 初始化内存
3. 允许中间 `l_ij` 用 register/cache 保留, 再做 reduce-max log-sum-exp → 提升数值稳定性和效率

## 🧠 数值稳定性的附加好处

通过提前设置:

text



Copy

🧠 数值稳定性的附加好处

通过提前设置：

textCopy

```
m_i = max_j (Q_i K_j^T)
l_i = sum_j exp(Q_i K_j^T - m_i)
```

FlashAttention 2 可以做到 **block-wise numerically stable softmax**，而不是每次都重新估算 max。

✅ 总结对比图

特性	FlashAttention 1	FlashAttention 2
Q 的加载位置	Inner loop	Outer loop
K/V 的加载位置	Outer loop	Inner loop
Q_i 重复读取	是	否，读一次
O_i/l_i 初始化位置	inner loop	outer loop
softmax 稳定性	less stable	more stable, blockwise numerics
整体性能	高	更高，少 memory IO



//-----

这个例子不理解，请具体说明，一步步解释

举个例子：

假设：

**Q 有 100 个 token**

**K/V 有 1000 个 token，按 100 token 为一个块**

**FlashAttention 1:**

每 10 次外循环中，**Q<sub>i</sub>** 要读 10 次

→ 读 **Q** 1000 次

**FlashAttention 2:**

每个 **Q<sub>i</sub>** 只读一次，然后循环访问不同的 **K/V<sub>j</sub>**

→ 读 **Q** 100 次（降低了 10 倍！）

好的，我们来一步一步解释这个例子，让你清楚理解为什么 FlashAttention 2 能减少对 Q 的重复读 Co

## ⚙️ 场景设定：

我们有如下张量大小（以简化讨论）：

- Query Q: shape = [100, d] (100 个 token)
- Key/Value K/V: shape = [1000, d] (1000 个 token)
- 我们采用 FlashAttention 分块机制：
  - 将 Q 按 100 tokens 分成 1 个 Q block (即 1 个  $Q_i$ )
  - 将 K/V 按 100 tokens 分成 10 个 K/V blocks (分别是  $K_1, K_2, \dots, K_{10}$ )

## 📦 FlashAttention 1：外层是 K/V，内层是 Q（错误做法）

伪代码如下：

```
python Copy 2
for j in range(10): # K/V blocks
 for i in range(1): # Q blocks
 Q_i = load(Q_block[i]) # 🔄 每轮内循环都读取一次 Q_i
 K_j = load(K_block[j]) ↓
 V_j = load(V_block[j])
```

## 📦 FlashAttention 1: 外层是 K/V, 内层是 Q (错误做法)

伪代码如下:

python

```
for j in range(10): # K/V blocks
 for i in range(1): # Q blocks
 Q_i = load(Q_block[i]) # 🔄 每轮内循环都读取一次 Q_i
 K_j = load(K_block[j])
 V_j = load(V_block[j])
 compute_attention(Q_i, K_j, V_j)
```

### 🛑 这里发生了什么?

- 外层循环: 10 个 K/V blocks (K\_1 到 K\_10)
- 每次外层循环, 内层都要对 Q\_i 做一次计算
- 虽然 Q 只有 1 个 block, 但每轮都重新从 memory 加载 Q\_i

🧮 计算:

- Q\_i 被重复读了 10 次 (每个 K\_j 要和 Q\_i 计算一次 attention)
- 所以总共 读取 Q\_i 10 次

## ✅ FlashAttention 2: 外层是 Q, 内层是 K/V (优化做法)

## ✅ FlashAttention 2: 外层是 Q, 内层是 K/V (优化做法)

伪代码如下:

python

```
for i in range(1): # Q blocks
 Q_i = load(Q_block[i]) # ✅ 只读取一次 Q_i
 for j in range(10): # K/V blocks
 K_j = load(K_block[j])
 V_j = load(V_block[j])
 compute_attention(Q_i, K_j, V_j)
```

### ✅ 这里发生了什么?

- 外层循环只有一次 (因为我们只有一个 Q block)
- 我们先读取 Q\_i 一次, 放在 **register / shared memory** 中
- 然后对每一个 K\_j 块进行 attention
- 不再重复读取 Q\_i!

🧮 计算:

- Q\_i 被读取 一次
- 总共只加载 Q 一次, 而不是十次!

🔍 具体数字对比总结



## 🔍 具体数字对比总结

模型版本	外层循环	Q_i 被读取的次数	总共读 Q 的次数
FlashAttention 1	10	每次都读取	10 次
FlashAttention 2	1	读取一次	1 次

## 🧠 为啥这很重要？

- GPU 的计算速度远远高于内存带宽 (memory-bound)
- 每次读 Q\_i 都会占用 memory 带宽
- 如果避免重复读取，就能腾出 memory 带宽给更重要的计算部分
- 🚀 这正是 FlashAttention 2 能做到速度提升、内存减少的核心优化！

如果你还想看一个更具体的 PyTorch/Pseudocode 示例（比如假装你是 kernel 里实现者），我可以继续展开。是否需要？

```
//-----
//
//-----
```

比较flash attention1 和flash attention2，为啥在flash attention2 中，

1. 要把Q从第二层循环放到第一层循环，把k, v从第一层循环放到第二层循环，
2. O\_i, I\_i可以从第二层的内层循环中，放到第一层和第二层之间？

flash attention 1 paper:

<https://arxiv.org/pdf/2205.14135>

flash attention 2 paper:

<https://arxiv.org/pdf/2307.08691>

To compare **FlashAttention-1** and **FlashAttention-2**, and to explain the specific changes in loop structure and variable placement you've asked about, I'll first provide a concise comparison of the two algorithms based on the provided papers ([FlashAttention-1](https://arxiv.org/pdf/2205.14135) and [FlashAttention-2](https://arxiv.org/pdf/2307.08691)). Then, I'll dive into the two specific questions about the loop reordering and variable placement, using insights from the papers and the algorithmic differences.

---

### ### Comparison of FlashAttention-1 and FlashAttention-2

#### #### FlashAttention-1 (FA-1)

- **Paper**: "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Aware Computation" (2022, arXiv:2205.14135)
- **Core Idea**: FA-1 is an IO-aware algorithm that reduces memory access by tiling (block-wise) computation of the attention mechanism, avoiding the need to materialize the full attention matrix  $(S = QK^T)$  or  $(A = \text{softmax}(S))$  in high-bandwidth memory (HBM). It uses:
  - **Tiling**: Splits  $(Q, K, V)$  into blocks, computing attention incrementally in SRAM.
  - **Recomputation**: Recomputes intermediate results (e.g.,  $(S)$ ) in the backward pass to save memory.
  - **Online Softmax**: Computes Softmax incrementally using per-row statistics  $(m_i, l_i)$ : maximum,  $(l_i)$ : sum) to avoid storing the full attention matrix.
- **Performance**: Achieves 2–4× speedup over standard attention on GPUs (e.g., A100), with significant memory savings.
- **Limitations**:
  - Suboptimal GPU utilization due to uneven workload distribution across thread blocks.
  - Limited kernel fusion, leading to multiple HBM accesses.
  - Less optimized for newer GPU architectures like Ampere (A100) or Hopper (H100).

#### #### FlashAttention-2 (FA-2)

- **Paper**: "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023, arXiv:2307.08691)
- **Core Idea**: FA-2 builds on FA-1, optimizing parallelism, work partitioning, and kernel fusion to better utilize modern GPUs (e.g., A100, H100). Key improvements include:
  - **Better Parallelism**: Reorders loops to maximize thread block utilization and reduce synchronization overhead.
  - **Work Partitioning**: Assigns warps to specific tasks (e.g., GEMM vs. Softmax) via warp-specialization, improving efficiency.
  - **Kernel Fusion**: Fuses multiple operations (e.g., scaling, masking, Softmax) into a single CUDA kernel, reducing HBM reads/writes.
  - **Low-Precision Support**: Optimizes for FP16/BF16, leveraging Tensor Cores for faster matrix multiplications.
- **Performance**: Achieves up to 2× speedup over FA-1, with higher GPU utilization (e.g., 72% SM occupancy on A100 vs. 35% for FA-1).
- **Key Differences**:
  - Loop reordering for better parallelism (addressed in Question 1).
  - Simplified variable updates by moving  $(O_i, l_i)$  placement (addressed in Question 2).
  - Reduced global synchronization and improved thread block scheduling.

---

### ### Question 1: Why Reorder Loops in FlashAttention-2?



In FlashAttention-2, the loop structure is changed such that:

- **Query (Q)** is moved from the second (inner) loop to the first (outer) loop.
- **Keys (K) and Values (V)** are moved from the first (outer) loop to the second (inner) loop.

Let's analyze the loop structures and the rationale for this change.

#### FlashAttention-1 Loop Structure

In FA-1 (Algorithm 1, page 17 of [FlashAttention-1 paper](https://arxiv.org/pdf/2205.14135)), the forward pass is structured as follows (simplified pseudocode):

```
python
FA-1 Forward Pass
for j in range(0, N, Bc): # Outer loop over K, V blocks (Bc = block size for columns)
 Load K_j, V_j into SRAM
 for i in range(0, N, Br): # Inner loop over Q blocks (Br = block size for rows)
 Load Q_i into SRAM
 S_ij = Q_i * K_j^T / sqrt(d)
 m_ij, l_ij, P_ij = OnlineSoftmax(S_ij)
 O_i = O_i * (l_i_old / l_i_new) + P_ij * V_j
 Update m_i, l_i
 Write O_i back to HBM
```

- **Outer Loop (j)**: Iterates over blocks of  $(K)$  and  $(V)$ , loading  $(K_j, V_j)$  into SRAM.
- **Inner Loop (i)**: Iterates over blocks of  $(Q)$ , loading  $(Q_i)$ , computing attention for block  $(i, j)$ , and updating output  $(O_i)$ .
- **Characteristics**:
  - $(K_j, V_j)$  are loaded once per outer loop and reused across all  $(Q_i)$ .
  - $(O_i, m_i, l_i)$  are updated incrementally in the inner loop, requiring frequent SRAM access.
- **Issue**: This structure leads to uneven workload distribution. The inner loop over  $(Q)$  requires multiple thread blocks to synchronize for  $(O_i)$  updates, causing stalls and reducing parallelism. The outer loop over  $(K, V)$  limits the number of independent tasks, underutilizing GPU thread blocks.

#### FlashAttention-2 Loop Structure

In FA-2 (Algorithm 1, page 16 of [FlashAttention-2 paper](https://arxiv.org/pdf/2307.08691)), the loop structure is reordered (simplified pseudocode):

```
python
FA-2 Forward Pass
for i in range(0, N, Br): # Outer loop over Q blocks
 Load Q_i into SRAM
 Initialize O_i, m_i, l_i in SRAM
 for j in range(0, N, Bc): # Inner loop over K, V blocks
 Load K_j, V_j into SRAM
 S_ij = Q_i * K_j^T / sqrt(d)
 m_ij, P_ij = OnlineSoftmax(S_ij)
 O_i += P_ij * V_j
 Update m_i, l_i
 O_i = O_i / l_i
 Write O_i back to HBM
```

- **Outer Loop (i)\*\*:** Iterates over blocks of  $(Q)$ , loading  $(Q_i)$  and initializing  $(O_i, m_i, l_i)$ .
- **Inner Loop (j)\*\*:** Iterates over blocks of  $(K, V)$ , loading  $(K_j, V_j)$ , computing attention for block  $(i, j)$ , and accumulating  $(O_i)$ .
- **Characteristics\*\*:**
  - $(Q_i)$  is loaded once per outer loop and reused across all  $(K_j, V_j)$ .
  - $(O_i, m_i, l_i)$  are initialized once per  $(Q_i)$  block and updated in the inner loop, with final normalization after the inner loop.
  - **Advantage\*\*:** Each  $(Q_i)$  block is processed independently in the outer loop, allowing more thread blocks to run in parallel without synchronization. The inner loop over  $(K, V)$  is compute-intensive, leveraging Tensor Cores efficiently.

#### #### Why Reorder Loops?

The reordering addresses several limitations in FA-1, as explained in Section 4.2 of the FA-2 paper:

##### 1. **Improved Parallelism\*\*:**

- In FA-1, the outer loop over  $(K, V)$  limits parallelism to  $(\lceil N / B_c \rceil)$  tasks, where  $(N)$  is sequence length and  $(B_c)$  is block size (e.g., 256). For large  $(N)$ , this is insufficient to saturate modern GPUs with thousands of threads.
- In FA-2, the outer loop over  $(Q)$  creates  $(\lceil N / B_r \rceil)$  independent tasks, where  $(B_r)$  is typically small (e.g., 128). This increases the number of parallel tasks, as each  $(Q_i)$  block can be assigned to a separate thread block, improving GPU occupancy (e.g., from 35% to 72% on A100).

##### 2. **Reduced Synchronization\*\*:**

- FA-1 requires frequent synchronization in the inner loop to update  $(O_i, m_i, l_i)$ , as multiple  $(Q_i)$  blocks contribute to the same output. This causes thread block stalls.
- FA-2 processes each  $(Q_i)$  block independently in the outer loop, reducing synchronization. The inner loop only accumulates results for a single  $(O_i)$ , minimizing inter-block dependencies.

##### 3. **Better Work Partitioning\*\*:**

- FA-2's loop structure aligns with warp-specialization (Section 4.3, FA-2 paper), where warps within a thread block are assigned specific tasks (e.g., GEMM for  $(QK^T)$ , Softmax for  $(P)$ ). The outer  $(Q)$  loop ensures each thread block focuses on a single  $(Q_i)$ , simplifying task allocation and reducing divergence.
- In FA-1, the outer  $(K, V)$  loop spreads work across multiple  $(Q_i)$ , complicating warp scheduling and reducing efficiency.

##### 4. **Memory Access Efficiency\*\*:**

- FA-2 loads  $(Q_i)$  once per outer loop, keeping it in SRAM for all  $(K_j, V_j)$  computations. This reduces HBM reads compared to FA-1, where  $(Q_i)$  is loaded repeatedly in the inner loop.
- The inner  $(K, V)$  loop in FA-2 streams  $(K_j, V_j)$  blocks, leveraging GPU's high memory bandwidth for sequential access.

---

#### ### Question 2: Why Move $(O_i, l_i)$ Placement in FlashAttention-2?

In FlashAttention-2, the variables  $(O_i)$  (output for row  $(i)$ ) and  $(l_i)$  (Softmax denominator for row  $(i)$ ) are moved from being updated in the inner loop of the second layer

(as in FA-1) to being initialized between the first and second loops and finalized after the second loop. Let's break this down.

#### #### FlashAttention-1 $\backslash(O_i, l_i \backslash)$ Placement

In FA-1 (Algorithm 1, page 17, FA-1 paper),  $\backslash(O_i, m_i, l_i \backslash)$  are updated incrementally within the inner loop:

```
```python
# FA-1 Inner Loop Snippet
for i in range(0, N, Br):
    Load Q_i
    m_i_old, l_i_old = m_i, l_i
    S_ij = Q_i * K_j^T / sqrt(d)
    m_ij = max(S_ij)
    P_ij = exp(S_ij - m_ij)
    l_ij = sum(P_ij)
    m_i = max(m_i_old, m_ij)
    l_i = l_i_old * exp(m_i_old - m_i) + l_ij
    O_i = O_i * (l_i_old / l_i) + P_ij * V_j
```
```

- **Placement**:  $\backslash(O_i, m_i, l_i \backslash)$  are updated for each block  $\backslash(i, j \backslash)$  in the inner loop over  $\backslash(Q \backslash)$ .

- **Update Logic**:

- $\backslash(m_i \backslash)$ : Tracks the maximum attention score for row  $\backslash(i \backslash)$ .
- $\backslash(l_i \backslash)$ : Tracks the Softmax denominator (sum of exponentials).
- $\backslash(O_i \backslash)$ : Accumulates the weighted sum  $\backslash(P_{ij} V_j \backslash)$ , scaled by the ratio of old and new denominators.

- **Issue**:

- Frequent updates to  $\backslash(O_i, l_i \backslash)$  in SRAM require multiple reads/writes, increasing memory traffic.
- The scaling factor  $\backslash(l_i^{\text{old}} / l_i \backslash)$  is computed repeatedly, adding computational overhead.
- Synchronization is needed to ensure consistent updates across thread blocks, reducing parallelism.

#### #### FlashAttention-2 $\backslash(O_i, l_i \backslash)$ Placement

In FA-2 (Algorithm 1, page 16, FA-2 paper),  $\backslash(O_i, l_i \backslash)$  are initialized before the inner loop and finalized after it:

```
```python
# FA-2 Forward Pass Snippet
for i in range(0, N, Br):
    Load Q_i
    O_i = zeros(Br, d) # Initialize O_i
    m_i = -inf         # Initialize m_i
    l_i = 0            # Initialize l_i
    for j in range(0, N, Bc):
        Load K_j, V_j
        S_ij = Q_i * K_j^T / sqrt(d)
        m_ij = max(S_ij)
        P_ij = exp(S_ij - m_ij)
        l_ij = sum(P_ij)
        m_i_new = max(m_i, m_ij)
```

```

    l_i = l_i * exp(m_i - m_i_new) + l_ij
    O_i += P_ij * V_j
    m_i = m_i_new
    O_i = O_i / l_i # Final normalization
    Write O_i to HBM
...

```

- **Placement**:

- **Initialization**: (O_i, m_i, l_i) are initialized once per (Q_i) block, before the inner loop over (K, V) .

- **Update**: (O_i, l_i) are accumulated in the inner loop without scaling.

- **Finalization**: (O_i) is normalized by (l_i) after the inner loop, before writing to HBM.

- **Update Logic**:

- (O_i) : Accumulates $(P_{ij} V_j)$ directly, without intermediate scaling.

- (l_i) : Accumulates the Softmax denominator incrementally.

- Normalization (O_i / l_i) is deferred to the end of the inner loop.

Why Move (O_i, l_i) ?

The placement change in FA-2 simplifies computation and improves efficiency, as detailed in Section 4.2 of the FA-2 paper:

1. **Simplified Computation**:

- In FA-1, (O_i) is scaled by (l_i^{old} / l_i) for each block, requiring extra multiplications and divisions.

- In FA-2, (O_i) is accumulated without scaling in the inner loop, and a single division by (l_i) is performed at the end. This reduces the number of operations per block, saving compute cycles.

2. **Reduced Memory Traffic**:

- FA-1's frequent updates to (O_i, l_i) in the inner loop increase SRAM read/write operations, as (O_i) is scaled and rewritten multiple times.

- FA-2 initializes (O_i, l_i) once and accumulates updates, reducing SRAM accesses. The final normalization is a single operation, minimizing memory traffic.

3. **Improved Kernel Fusion**:

- By deferring normalization, FA-2 allows more operations (e.g., GEMM for $(P_{ij} V_j)$, Softmax updates) to be fused into a single CUDA kernel. This reduces HBM read/write cycles compared to FA-1, where scaling (O_i) breaks fusion opportunities.

- The FA-2 paper notes that kernel fusion (Section 4.4) eliminates redundant HBM accesses, contributing to the 2× speedup.

4. **Better Parallelism**:

- In FA-1, updating (O_i, l_i) in the inner loop requires synchronization across threads processing different (Q_i) , as (O_i) is shared.

- In FA-2, each (Q_i) block is independent in the outer loop, and (O_i, l_i) are local to that block. This reduces synchronization overhead, allowing thread blocks to execute more independently.

5. **Numerical Stability**:

- FA-2's deferred normalization maintains numerical stability by accumulating (l_i) in FP16/BF16 and performing a single division at the end, leveraging the online Softmax algorithm's robustness (inherited from FA-1 but simplified).

Summary

- **Loop Reordering (Question 1)**:**

- FA-2 moves (Q) to the outer loop and (K, V) to the inner loop to increase parallelism, reduce synchronization, and improve work partitioning.

- This allows more thread blocks to process independent (Q_i) blocks, boosting GPU occupancy (e.g., 72% vs. 35%) and reducing stalls.

- **(O_i, L_i) Placement (Question 2)**:**

- FA-2 initializes (O_i, L_i) before the inner loop and normalizes (O_i) after it, simplifying computation, reducing memory traffic, and enabling better kernel fusion.

- This eliminates frequent scaling operations, minimizes SRAM accesses, and enhances parallelism by isolating each (Q_i) block's updates.

These changes make FA-2 significantly faster and more efficient than FA-1, especially on modern GPUs like A100 and H100, as validated by the 2× speedup reported in the FA-2 paper.

If you need a detailed code comparison (e.g., PyTorch or CUDA implementations) or a visual flowchart to illustrate the loop structures, please let me know!

Copy

Comparison of FlashAttention-1 and FlashAttention-2

FlashAttention-1 (FA-1)

- **Paper:** "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Aware Computation" (2022, arXiv:2205.14135)
- **Core Idea:** FA-1 is an IO-aware algorithm that reduces memory access by tiling (block-wise) computation of the attention mechanism, avoiding the need to materialize the full attention matrix $S = QK^T$ or $A = \text{softmax}(S)$ in high-bandwidth memory (HBM). It uses:
 - **Tiling:** Splits Q, K, V into blocks, computing attention incrementally in SRAM.
 - **Recomputation:** Recomputes intermediate results (e.g., S) in the backward pass to save memory.
 - **Online Softmax:** Computes Softmax incrementally using per-row statistics (m_i : maximum, l_i : sum) to avoid storing the full attention matrix.
- **Performance:** Achieves 2–4× speedup over standard attention on GPUs (e.g., A100), with significant memory savings.
- **Limitations:**
 - Suboptimal GPU utilization due to uneven workload distribution across thread blocks.
 - Limited kernel fusion, leading to multiple HBM accesses.
 - Less optimized for newer GPU architectures like Ampere (A100) or Hopper (H100).

FlashAttention-2 (FA-2)

- **Paper:** "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023, arXiv:2307.08691)

Copy

FlashAttention-2 (FA-2)

- **Paper:** "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023, arXiv:2307.08691)
- **Core Idea:** FA-2 builds on FA-1, optimizing parallelism, work partitioning, and kernel fusion to better utilize modern GPUs (e.g., A100, H100). Key improvements include:
 - **Better Parallelism:** Reorders loops to maximize thread block utilization and reduce synchronization overhead.
 - **Work Partitioning:** Assigns warps to specific tasks (e.g., GEMM vs. Softmax) via warp-specialization, improving efficiency.
 - **Kernel Fusion:** Fuses multiple operations (e.g., scaling, masking, Softmax) into a single CUDA kernel, reducing HBM reads/writes.
 - **Low-Precision Support:** Optimizes for FP16/BF16, leveraging Tensor Cores for faster matrix multiplications.
- **Performance:** Achieves up to 2× speedup over FA-1, with higher GPU utilization (e.g., 72% SM occupancy on A100 vs. 35% for FA-1).
- **Key Differences:**
 - Loop reordering for better parallelism (addressed in Question 1).
 - Simplified variable updates by moving O_i, l_i placement (addressed in Question 2).
 - Reduced global synchronization and improved thread block scheduling.

Question 1: Why Reorder Loops in FlashAttention-2?



Copy

Question 1: Why Reorder Loops in FlashAttention-2?

In FlashAttention-2, the loop structure is changed such that:

- **Query (Q)** is moved from the second (inner) loop to the first (outer) loop.
- **Keys (K) and Values (V)** are moved from the first (outer) loop to the second (inner) loop.

Let's analyze the loop structures and the rationale for this change.

FlashAttention-1 Loop Structure

In FA-1 (Algorithm 1, page 17 of [FlashAttention-1 paper](#)), the forward pass is structured as follows (simplified pseudocode):

python

✕ Collapse

≡ Wrap

▶ Run

📄 C

```
# FA-1 Forward Pass
for j in range(0, N, Bc): # Outer loop over K, V blocks (Bc = block size for columns)
    Load K_j, V_j into SRAM
    for i in range(0, N, Br): # Inner loop over Q blocks (Br = block size for rows)
        Load Q_i into SRAM
        S_ij = Q_i * K_j^T / sqrt(d)
        m_ij, l_ij, P_ij = OnlineSoftmax(S_ij)
        O_i = O_i * (l_i_old / l_i_new) + P_ij * V_j
        Update m_i, l_i
    Write O_i back to HBM
```

- **Outer Loop (j):** Iterates over blocks of K and V , loading K_j, V_j into SRAM.
- **Inner Loop (i):** Iterates over blocks of Q , loading Q_i , computing attention for block (i, j) , and updating output O_i .

Copy

- **Outer Loop (j):** Iterates over blocks of K and V , loading K_j, V_j into SRAM.
- **Inner Loop (i):** Iterates over blocks of Q , loading Q_i , computing attention for block (i, j) , and updating output O_i .
- **Characteristics:**
 - K_j, V_j are loaded once per outer loop and reused across all Q_i .
 - O_i, m_i, l_i are updated incrementally in the inner loop, requiring frequent SRAM access.
 - **Issue:** This structure leads to uneven workload distribution. The inner loop over Q requires multiple thread blocks to synchronize for O_i updates, causing stalls and reducing parallelism. The outer loop over K, V limits the number of independent tasks, underutilizing GPU thread blocks.

FlashAttention-2 Loop Structure

In FA-2 (Algorithm 1, page 16 of [FlashAttention-2 paper](#)), the loop structure is reordered (simplified pseudocode):

```
python ✕ Collapse  ⇅ Wrap  ▶ Run  📄 Copy
```

```
# FA-2 Forward Pass
for i in range(0, N, Br): # Outer loop over Q blocks
    Load Q_i into SRAM
    Initialize O_i, m_i, l_i in SRAM
    for j in range(0, N, Bc): # Inner loop over K, V blocks
        Load K_j, V_j into SRAM
        S_ij = Q_i * K_j^T / sqrt(d)
        m_ij, P_ij = OnlineSoftmax(S_ij)
        O_i += P_ij * V_j
        Update m_i, l_i
    O_i = O_i / l_i
```


Copy Python

Collapse

Wrap

Run

Copy

```
# FA-2 Forward Pass
for i in range(0, N, Br): # Outer loop over Q blocks
    Load Q_i into SRAM
    Initialize O_i, m_i, l_i in SRAM
    for j in range(0, N, Bc): # Inner loop over K, V blocks
        Load K_j, V_j into SRAM
        S_ij = Q_i * K_j^T / sqrt(d)
        m_ij, P_ij = OnlineSoftmax(S_ij)
        O_i += P_ij * V_j
        Update m_i, l_i
    O_i = O_i / l_i
    Write O_i back to HBM
```

- **Outer Loop (i):** Iterates over blocks of Q , loading Q_i and initializing O_i, m_i, l_i .
- **Inner Loop (j):** Iterates over blocks of K, V , loading K_j, V_j , computing attention for block (i, j) , and accumulating O_i .
- **Characteristics:**
 - Q_i is loaded once per outer loop and reused across all K_j, V_j .
 - O_i, m_i, l_i are initialized once per Q_i block and updated in the inner loop, with final normalization after the inner loop.
 - **Advantage:** Each Q_i block is processed independently in the outer loop, allowing more thread blocks to run in parallel without synchronization. The inner loop over K, V is compute-intensive, leveraging Tensor Cores efficiently.

Why Reorder Loops?





Copy

Why Reorder Loops?

The reordering addresses several limitations in FA-1, as explained in Section 4.2 of the FA-2 paper:

1. Improved Parallelism:

- In FA-1, the outer loop over K, V limits parallelism to $\lceil N/B_c \rceil$ tasks, where N is sequence length and B_c is block size (e.g., 256). For large N , this is insufficient to saturate modern GPUs with thousands of threads.
- In FA-2, the outer loop over Q creates $\lceil N/B_r \rceil$ independent tasks, where B_r is typically small (e.g., 128). This increases the number of parallel tasks, as each Q_i block can be assigned to a separate thread block, improving GPU occupancy (e.g., from 35% to 72% on A100).

2. Reduced Synchronization:

- FA-1 requires frequent synchronization in the inner loop to update O_i, m_i, l_i , as multiple Q_i blocks contribute to the same output. This causes thread block stalls.
- FA-2 processes each Q_i block independently in the outer loop, reducing synchronization. The inner loop only accumulates results for a single O_i , minimizing inter-block dependencies.

3. Better Work Partitioning:

- FA-2's loop structure aligns with warp-specialization (Section 4.3, FA-2 paper), where warps within a thread block are assigned specific tasks (e.g., GEMM for QK^T , Softmax for P). The outer Q loop ensures each thread block focuses on a single Q_i , simplifying task allocation and reducing divergence.
- In FA-1, the outer K, V loop spreads work across multiple Q_i , complicating warp scheduling and reducing efficiency.

4. Memory Access Efficiency:





4. Memory Access Efficiency:

Copy

- FA-2 loads Q_i once per outer loop, keeping it in SRAM for all K_j, V_j computations. This reduces HBM reads compared to FA-1, where Q_i is loaded repeatedly in the inner loop.
- The inner K, V loop in FA-2 streams K_j, V_j blocks, leveraging GPU's high memory bandwidth for sequential access.

Question 2: Why Move O_i, l_i Placement in FlashAttention-2?

In FlashAttention-2, the variables O_i (output for row i) and l_i (Softmax denominator for row i) are moved from being updated in the inner loop of the second layer (as in FA-1) to being initialized between the first and second loops and finalized after the second loop. Let's break this down.

FlashAttention-1 O_i, l_i Placement

In FA-1 (Algorithm 1, page 17, FA-1 paper), O_i, m_i, l_i are updated incrementally within the inner loop:

python

✖ Collapse

≡ Wrap

▶ Run

📄 Copy

```
# FA-1 Inner Loop Snippet
for i in range(0, N, Br):
    Load Q_i
    m_i_old, l_i_old = m_i, l_i
    S_ij = Q_i * K_j^T / sqrt(d)
    m_ij = max(S_ij)
    P_ij = exp(S_ij - m_ij)
    l_ij = sum(P_ij)
    m_i = max(m_i_old, m_ij)
    l_i = l_i_old * exp(m_i_old - m_i) + l_ij
    O_i = O_i * (l_i_old / l_i) + P_ij * V_j
```



```
l_i = l_i_old * exp(m_i_old - m_i) + l_ij
O_i = O_i * (l_i_old / l_i) + P_ij * V_j
```

✖ Collapse

≡ Wrap

▶ Run

📄 Copy

- **Placement:** O_i, m_i, l_i are updated for each block (i, j) in the inner loop over Q .
- **Update Logic:**
 - m_i : Tracks the maximum attention score for row i .
 - l_i : Tracks the Softmax denominator (sum of exponentials).
 - O_i : Accumulates the weighted sum $P_{ij}V_j$, scaled by the ratio of old and new denominators.
- **Issue:**
 - Frequent updates to O_i, l_i in SRAM require multiple reads/writes, increasing memory traffic.
 - The scaling factor l_{i_old} / l_i is computed repeatedly, adding computational overhead.
 - Synchronization is needed to ensure consistent updates across thread blocks, reducing parallelism.

FlashAttention-2 O_i, l_i Placement

In FA-2 (Algorithm 1, page 16, FA-2 paper), O_i, l_i are initialized before the inner loop and finalized after it:

python

✖ Collapse

≡ Wrap

▶ Run

📄 Copy

```
# FA-2 Forward Pass Snippet
for i in range(0, N, Br):
    Load Q_i
    O_i = zeros(Br, d) # Initialize O_i
    m_i = -inf         # Initialize m_i
    l_i = 0            # Initialize l_i
    for j in range(0, N, Bc):
        Load K_i, V_i
```



python

Collapse

Wrap

Run

Copy

Copy

```
# FA-2 Forward Pass Snippet
for i in range(0, N, Br):
    Load Q_i
    O_i = zeros(Br, d) # Initialize O_i
    m_i = -inf         # Initialize m_i
    l_i = 0            # Initialize l_i
    for j in range(0, N, Bc):
        Load K_j, V_j
        S_ij = Q_i * K_j^T / sqrt(d)
        m_ij = max(S_ij)
        P_ij = exp(S_ij - m_ij)
        l_ij = sum(P_ij)
        m_i_new = max(m_i, m_ij)
        l_i = l_i * exp(m_i - m_i_new) + l_ij
        O_i += P_ij * V_j
        m_i = m_i_new
    O_i = O_i / l_i # Final normalization
    Write O_i to HBM
```

- **Placement:**
 - **Initialization:** O_i, m_i, l_i are initialized once per Q_i block, before the inner loop over K, V .
 - **Update:** O_i, l_i are accumulated in the inner loop without scaling.
 - **Finalization:** O_i is normalized by l_i after the inner loop, before writing to HBM.
- **Update Logic:**
 - O_i : Accumulates $P_{ij}V_j$ directly, without intermediate scaling.
 - l_i : Accumulates the Softmax denominator incrementally.

by

- **Finalization:** O_i is normalized by l_i after the inner loop, before writing to HBM.
- **Update Logic:**
 - O_i : Accumulates $P_{ij}V_j$ directly, without intermediate scaling.
 - l_i : Accumulates the Softmax denominator incrementally.
 - Normalization (O_i/l_i) is deferred to the end of the inner loop.

Why Move O_i, l_i ?

The placement change in FA-2 simplifies computation and improves efficiency, as detailed in Section 4.2 of the FA-2 paper:

1. Simplified Computation:

- In FA-1, O_i is scaled by l_i^{old} / l_i for each block, requiring extra multiplications and divisions.
- In FA-2, O_i is accumulated without scaling in the inner loop, and a single division by l_i is performed at the end. This reduces the number of operations per block, saving compute cycles.

2. Reduced Memory Traffic:

- FA-1's frequent updates to O_i, l_i in the inner loop increase SRAM read/write operations, as O_i is scaled and rewritten multiple times.
- FA-2 initializes O_i, l_i once and accumulates updates, reducing SRAM accesses. The final normalization is a single operation, minimizing memory traffic.

3. Improved Kernel Fusion:

- By deferring normalization, FA-2 allows more operations (e.g., GEMM for $P_{ij}V_j$, Softmax updates) to be fused into a single CUDA kernel. This reduces HBM read/write cycles compared to FA-1, where scaling O_i breaks fusion opportunities.

3. Improved Kernel Fusion:

- By deferring normalization, FA-2 allows more operations (e.g., GEMM for $P_{ij}V_j$, Softmax updates) to be fused into a single CUDA kernel. This reduces HBM read/write cycles compared to FA-1, where scaling O_i breaks fusion opportunities.
- The FA-2 paper notes that kernel fusion (Section 4.4) eliminates redundant HBM accesses, contributing to the 2× speedup.

4. Better Parallelism:

- In FA-1, updating O_i, l_i in the inner loop requires synchronization across threads processing different Q_i , as O_i is shared.
- In FA-2, each Q_i block is independent in the outer loop, and O_i, l_i are local to that block. This reduces synchronization overhead, allowing thread blocks to execute more independently.

5. Numerical Stability:

- FA-2's deferred normalization maintains numerical stability by accumulating l_i in FP16/BF16 and performing a single division at the end, leveraging the online Softmax algorithm's robustness (inherited from FA-1 but simplified).

Summary

- **Loop Reordering (Question 1):**
 - FA-2 moves Q to the outer loop and K, V to the inner loop to increase parallelism, reduce synchronization, and improve work partitioning.
 - This allows more thread blocks to process independent Q_i blocks, boosting GPU occupancy (e.g., 72% vs. 35%) and reducing stalls.
- **O_i, l_i Placement (Question 2):**



5. Numerical Stability:

- FA-2's deferred normalization maintains numerical stability by accumulating l_i in FP16/BF16 and performing a single division at the end, leveraging the online Softmax algorithm's robustness (inherited from FA-1 but simplified).

Summary

- **Loop Reordering (Question 1):**
 - FA-2 moves Q to the outer loop and K, V to the inner loop to increase parallelism, reduce synchronization, and improve work partitioning.
 - This allows more thread blocks to process independent Q_i blocks, boosting GPU occupancy (e.g., 72% vs. 35%) and reducing stalls.
- **O_i, l_i Placement (Question 2):**
 - FA-2 initializes O_i, l_i before the inner loop and normalizes O_i after it, simplifying computation, reducing memory traffic, and enabling better kernel fusion.
 - This eliminates frequent scaling operations, minimizes SRAM accesses, and enhances parallelism by isolating each Q_i block's updates.

These changes make FA-2 significantly faster and more efficient than FA-1, especially on modern GPUs like A100 and H100, as validated by the 2× speedup reported in the FA-2 paper.

If you need a detailed code comparison (e.g., PyTorch or CUDA implementations) or a visual flowchart to illustrate the loop structures, please let me know!

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----

//-----

//-----

//-----

//-----

//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----

//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----

//-----
//
//-----